
A PROBABILISTIC CHOREOGRAPHY LANGUAGE FOR PRISM

MARCO CARBONE ^a AND ADELE VESCHETTI ^b

^a Computer Science Department, IT University of Copenhagen, Rued Langgaards Vej 7, 2300 Copenhagen S, Denmark
e-mail address: carbonem@itu.dk, maca@itu.dk

^b Department of Computer Science, TU Darmstadt, Hochschulstraße 10, 64289 Darmstadt, Germany
e-mail address: adele.veschetti@tu-darmstadt.de

ABSTRACT. We present a choreographic framework for modelling and analysing concurrent probabilistic systems based on the PRISM model-checker. This is achieved through the development of a choreography language, which is a specification language that allows to describe the desired interactions within a concurrent system from a global viewpoint. Using choreographies provides a global view of system interactions, which can help in understanding process flow and identifying potential modelling issues. We equip our language with a probabilistic semantics and then define a formal encoding into the PRISM language and discuss its correctness. Properties of programs written in our choreographic language can be model-checked by the PRISM model-checker via their translation into the PRISM language. Finally, we implement a compiler for our language and demonstrate its practical applicability via examples drawn from use cases featured in the PRISM website.

1. INTRODUCTION

Programming distributed systems presents significant challenges due to their inherent complexity, and the possibility of obscure edge cases arising from their complex interactions. Unlike monolithic systems, distributed programs involve multiple nodes operating concurrently and communicating over networks, introducing a multitude of potential failure scenarios and nondeterministic behaviours. One of the primary challenges in understanding distributed systems lies in the fact that the interactions between multiple components can diverge from the sum of their individual behaviours. This emergent behaviour often results from subtle interactions between nodes, making it difficult to predict and reason about a system's overall behaviour.

PRISM [1] is a probabilistic model checker that offers a specialised language for the specification and verification of probabilistic concurrent systems. PRISM has been used in various fields, including multimedia protocols [21], randomised distributed algorithms [18, 20], security protocols [25, 19], and biological systems [12, 15]. At its core, PRISM provides a declarative language with a set of constructs for describing probabilistic behaviours and properties within a system. Given a distributed system, we can use PRISM to model the

Key words and phrases: choreographic language, PRISM, probabilistic systems modelling.

behaviour of each of its nodes, and then verify desired properties for the entire system. However, this approach can become difficult to manage as the number of nodes increases.

Choreographic programming [22] is an emerging programming paradigm for distributed systems in which programs, referred to as choreographies, specify communication protocols from a global perspective. Instead of programming the behaviour of each component independently or relying on a central orchestrator, choreographic languages describe the interactions among participants directly. A choreography therefore abstracts away from the internal implementation of individual components and focuses on the communication structure that governs their coordination. In this approach, the primary concern is not the arbitrary interleavings of local computations but the communication structure of the system: which participants interact, in which order, and under which conditions. Choreographies make these interaction patterns explicit at the level of the program. Choreographic programming naturally applies to distributed systems whose behaviour can be described through structured interaction protocols. Systems whose dynamics rely heavily on loosely coordinated, highly unstructured, or unpredictable communication patterns are generally less amenable to this approach. Within its intended scope, however, choreographic programming has proved effective in practice, since choreographies enable the automatic generation of decentralised implementations that are correct by construction [8].

This paper presents a choreographic language designed for modelling concurrent probabilistic systems and introduces a *projection* function for translating protocols described in this language into PRISM code. This choreographic approach not only simplifies the modelling process but also ensures integration with PRISM's powerful analysis capabilities. In order to illustrate the idea concretely, consider the following choreography:

$$C \stackrel{\text{def}}{=} \mathbf{p} \rightarrow \{\mathbf{q}\} \begin{cases} \lambda_1 : (x' = 1) \ \& \ (y' = 2); \ C \\ \lambda_2 : (x' = 3) \ \& \ (y' = 1); \ C \end{cases}$$

This specification recursively defines an interaction where process $\mathbf{p}$ communicates with process $\mathbf{q}$, choosing the first branch with rate λ_1 (and setting x to 1 and y to 2) or the second branch with rate λ_2 (and setting x to 3 and y to 1). Here, we adopt PRISM notation where the prime symbol, e.g., x' , y' , indicates the next-state value of the corresponding variable, that is, the value it will take after the transition occurs. The projection mechanism translates this global choreography into local PRISM modules by annotating each branch with a unique label (e.g., a and b) and using a state variable (e.g., $s_{\mathbf{p}}$ and $s_{\mathbf{q}}$) to uniquely identify each step. One possible representation of the behaviour described by the choreography in PRISM is the following:

$$\begin{aligned} \mathbf{p} : \{ & [a] \ s_{\mathbf{p}} = 0 \rightarrow \lambda_1 : (x' = 1) \ \& \ (s'_{\mathbf{p}} = 1) & [] \ s_{\mathbf{p}} = 1 \rightarrow 1 : (s'_{\mathbf{p}} = 0) \\ & [b] \ s_{\mathbf{p}} = 0 \rightarrow \lambda_2 : (x' = 3) \ \& \ (s'_{\mathbf{p}} = 2) & [] \ s_{\mathbf{p}} = 2 \rightarrow 1 : (s'_{\mathbf{p}} = 0) \} \\ \mathbf{q} : \{ & [a] \ s_{\mathbf{q}} = 0 \rightarrow 1 : (y' = 2) \ \& \ (s'_{\mathbf{q}} = 1) & [] \ s_{\mathbf{q}} = 1 \rightarrow 1 : (s'_{\mathbf{q}} = 0) \\ & [b] \ s_{\mathbf{q}} = 0 \rightarrow 1 : (y' = 1) \ \& \ (s'_{\mathbf{q}} = 2) & [] \ s_{\mathbf{q}} = 2 \rightarrow 1 : (s'_{\mathbf{q}} = 0) \} \end{aligned}$$

In this translation, modules $\mathbf{p}$ and $\mathbf{q}$ contain a set of commands. For example, $[a] \ s_{\mathbf{p}} = 0 \rightarrow \lambda_1 : (x' = 1) \ \& \ (s'_{\mathbf{p}} = 1)$ in module $\mathbf{p}$ is a command where a is a label over which the module can synchronise with other modules, $s_{\mathbf{p}} = 0$ is a guard, λ_1 is a rate, and $(x' = 1) \ \& \ (s'_{\mathbf{p}} = 1)$ is a list of commands for modifying the state. Here, we use labels a and b for simulating the two branches of the choreography. The command labeled a in $\mathbf{p}$'s module corresponds to

the branch with rate λ_1 , and the matching command in $\mathbf{q}$'s module (with rate 1) sets y to 2; similarly, label b corresponds to the second branch with rate λ_2 for $\mathbf{p}$ and rate 1 for $\mathbf{q}$, setting x and y appropriately. After executing an interaction command in each module, a subsequent reset command (with rate 1) sets the state counters $s_{\mathbf{p}}$ and $s_{\mathbf{q}}$ back to 0 due to the recursive call following the branching. This mechanism ensures that these commands are only executable when the system has reached this particular state.

Through our contributions, we aim to provide a smooth workflow for modelling, analysing, and verifying concurrent probabilistic systems, ultimately increasing their usability in various application domains. In particular, by employing choreographies, we gain a clear and comprehensive view of the interactions occurring within the system, allowing us to discern the flow of processes and detect any potential sources of error in the modelling phase.

Contributions and Overview. We summarise our contributions as follows:

- we propose a choreographic language equipped with well-defined syntax and semantics, tailored specifically for describing concurrent systems with probabilistic behaviours (§ 3);
- we introduce a semantics for the minimal fragment of PRISM needed for code generation from choreographies (§ 4), which follows PRISM's original semantics [1];
- we establish a rigorous definition for a translation function from choreographies to PRISM (§ 5), and address its correctness. This translation serves as a crucial intermediary step in transforming models described in our choreographic language into PRISM-compatible representations;
- we give an implementation [2] that executes the defined translation function, enabling users to utilise PRISM's robust analysis features while benefiting from the expressiveness of choreographies;
- we show some use cases in order to demonstrate the applicability of our language (§ 6).

Changes with respect to the conference version. This article is an expanded version of our paper presented at COORDINATION 2024 [9]. It includes detailed definitions and a complete proof of the main theorem, which were previously omitted. We also present a revised projection function that provides a cleaner formalisation and no longer relies on counters for the control state. Furthermore, we introduce additional language constructs, which are also supported by the implementation. Additionally, we provide a more extensive discussion of the language features supported by the implementation and expand the related work section. Finally, we introduce new use cases analysed through choreographies, including case studies on a simple peer-to-peer protocol, synchronous leader election, and the Dining Cryptographers protocol.

2. A MOTIVATING EXAMPLE

As an example, we consider a simplified version of the thinkteam example, a three-tier data management system [28], presented in the PRISM documentation¹.

```

1      ctmc
2      module User
3          User.STATE : [0..2] init 0;
4
5          [alpha_1] (User.STATE=0)  $\rightarrow$  lambda : (User.STATE'=1);

```

¹<https://www.prismmodelchecker.org/casestudies/thinkteam.php>

```

6      [alpha_2] (User.STATE=0) → lambda : (User.STATE'=2);
7      [beta] (User.STATE=1) → mu : (User.STATE'=0);
8      [gamma_1] (User.STATE=2) → theta : (User.STATE'=1);
9      [gamma_2] (User.STATE=2) → theta : (User.STATE'=2);
10     endmodule
11
12     module CheckOut
13         CheckOut.STATE : [0..1] init 0;
14
15         [alpha_1, alpha_2] (CheckOut.STATE=0) → 1 : (CheckOut.STATE'=1);
16         [beta] (CheckOut.STATE=1) → 1 : (CheckOut.STATE'=0);
17         [gamma_1, gamma_2] (CheckOut.STATE=1) → 1 : (CheckOut.STATE'=1);
18     endmodule

```

The protocol aims to manage exclusive access to a single file, controlled by the `CheckOut` process. Users can request access, which can be granted based on the file's current status. The goal is to ensure that only one user possesses the file at any given time while allowing for efficient access requests and retries in case of denial. Users move between different states (0, 1, or 2) based on the granted or denied access to the file with corresponding rates `lambda`, `mu`, and `theta`; the `CheckOut` process transitions between two states (0 or 1):

In PRISM, modules are individual processes whose behaviour is specified by a collection of commands, in a declarative fashion. Processes have a local state, can interact with other modules and query each other's state. Above, the modules `User` (lines 2-10) and `CheckOut` (lines 12-18) can synchronise on different labels, e.g., `alpha_1`. On line 5, `(User.STATE=0)` is a condition indicating that this transition is enabled when `User.STATE` has value 0. The variable `lambda` is a rate, since the program models a Continuous Time Markov Chain (CTMC). The command `(User.STATE'=1)` is an update, indicating that `User.STATE` changes to 1 when this transition fires.

Understanding the interactions between processes in this example may require inspecting several module transitions. When formalised using our choreographic language, the interaction structure is expressed as a single global description, which makes the communication pattern directly visible.

```

C0 := CheckOut → User : (+["1*lambda"] ; C1 +["1*lambda"] ; C2)
C1 := CheckOut → User : (+["1*theta"] ; C0)
C2 := CheckOut → User : (+["1*mu"] ; C1 +["1*mu"] ; C2)

```

In this model, we define three distinct choreographies, namely `C0`, `C1`, and `C2`. These choreographies describe the interaction patterns between the modules `CheckOut` and `User`. The state updates resulting from these interactions are not explicitly depicted as they are not relevant for this particular protocol, but necessary in the PRISM implementation. As evident from this example, the choreographic language facilitates a straightforward understanding of the interactions between processes, minimizing the likelihood of errors. In fact, we can think of the choreography representation of this example as the product of the two PRISM modules seen above.

In order to formally analyze the system, we need to project the choreography onto individual components, resulting in a PRISM model that captures the behaviour of each process explicitly. The projection process ensures that each role in the choreography, such as `User` and `CheckOut`, is assigned a state variable that tracks its execution progress. Interactions in the choreography correspond to PRISM synchronisation labels, ensuring

consistency across transitions. Furthermore, the rates in the choreography, such as **lambda**, **mu**, and **theta**, translate into transition rates in the PRISM model.

Each choreographic rule maps to a corresponding PRISM command. For instance, the choreographic transition

`C0 := CheckOut → User : (+["1*lambda"] ; C1)`

corresponds to the following PRISM code:

```

1 ...
2 [alpha_1] (User.STATE=0) -> lambda : (User.STATE'=1); // User module
3 ...
4 [alpha_1] (CheckOut.STATE=0) -> 1 : (CheckOut.STATE'=1); // CheckOut module
5 ...

```

This ensures both modules execute in sync under the label `alpha_1` and only when the modules are in the correct states (`CheckOut.STATE=0` and `User.STATE=0`). Similar transformations apply to other interactions.

3. CHOREOGRAPHY LANGUAGE

In the previous section, we illustrated our choreography language by means of an example. We now formalise the language by presenting its syntax and semantics. For the sake of clarity, we slightly depart from the concrete syntax of our tool, adopting instead a more process-algebraic format, which makes the syntax more concise and better suited to formal definitions and proofs.

3.1. Syntax. Let **p** range over a (possibly infinite) set of module names $\mathcal{R}$, x over a (possibly infinite) set of variables Var , and v over a (possibly infinite) set of values Val . Choreographies, the key component of our language, are defined by the following syntax:

$$\begin{array}{lll}
C & ::= & \mathbf{p} \rightarrow \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \Sigma_{j \in J} \lambda_j : u_j; C_j \quad (\text{interaction}) \\
& | & \text{if } E @ \mathbf{p} \text{ then } C_1 \text{ else } C_2 \quad (\text{conditional}) \\
& | & X \quad (\text{recursive call}) \\
& | & \mathbf{0} \quad (\text{inact}) \\
\\
\mathcal{D} & ::= & X \stackrel{\text{def}}{=} C, \mathcal{D} \quad | \quad \emptyset \quad (\text{definitions}) \\
u & ::= & (x' = E) \ \& \ u \quad | \quad (x' = E) \quad (\text{assignments}) \\
E, g & ::= & f(\tilde{E}) \quad | \quad x \quad | \quad v \quad (\text{expressions})
\end{array}$$

The syntactic category C denotes choreographic programs. The interaction term $\mathbf{p} \rightarrow \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \Sigma\{\lambda_j : x_j = E_j. C_j\}_{j \in J}$ denotes an interaction initiated by module $\mathbf{p}$ with modules $\mathbf{p}_i$'s, with $\mathbf{p}$ and $\mathbf{p}_i$'s all distinct. A choreography specifies what interaction must be executed next, shifting the focus from what can happen to what must happen. When the interaction happens, one of the j branches is selected as a continuation. Branching is a random move: the number $\lambda_j \in \mathbb{R}$ denotes either a probability or a rate. This will depend on the language we wish to use. In the case of probabilities, it must be the case that $0 \leq \lambda_j \leq 1$ and $\Sigma_j \lambda_j = 1$. Once a branch j is taken, the choreography will execute some assignments u_j . A single assignment has the syntax $(x' = E)$ meaning that the value obtained by evaluating expression E is assigned to variable x ; assignments can be concatenated with the operator $\&$. Note that x' is used for an assignment to x : here, we follow the syntax adopted in

PRISM (see § 4). Expressions are obtained by applying some unspecified functions to other expressions or, as base terms, i.e., variables and values (denoted by v).

The term **if** $E@p$ **then** C_1 **else** C_2 denotes a system where module p evaluates the guard E (which can contain variables located at other modules) and then (deterministically) branches accordingly. The term X is a (possibly recursive) procedure call: in the semantics, we assume that such procedure names are defined separately. Moreover, we assume that such definitions are non-contractive, i.e., we avoid definitions of the form $X \stackrel{\text{def}}{=} X$. The term **0** denotes the system finishing its computation.

Assumption 1. *We assume that every writable choreography is such that, for every interaction $p \rightarrow \{p_1, \dots, p_n\} : \Sigma_{j \in J} \lambda_j : u_j$; C_j , all updates in each u_j are assignments to variables located at $p, p_1, \dots, p_n$.*

3.2. Semantics. The semantics of a choreography is a relation that captures how the values assigned to variables are modified when the various modules synchronise with each other. Similar to the operational semantics of imperative languages, we define a state, denoted by S , as a mapping from variables to values, that is, $S : \text{Var} \rightarrow \text{Val}$.

Given a state, substitution allows us to modify some of its values.

Definition 1. *Given distinct variables $x_1, \dots, x_n$ and expressions $E_1, \dots, E_n$, a state update $S[x'_1 = E_1 \ \& \ \dots \ \& \ x'_n = E_n]$ is defined as*

$$S[x'_1 = E_1 \ \& \ \dots \ \& \ x'_n = E_n](y) = \begin{cases} E_i \downarrow_S & \text{if } y = x_i \text{ for some } i \in \{1, \dots, n\}, \\ S(y) & \text{otherwise.} \end{cases}$$

where $E_i \downarrow_S$ denotes a (decidable) evaluation of the expression E in the state S .

Let $\mathcal{S}$ be the set of all possible states and let $\mathcal{D}$ be a set of definitions of the form $X \stackrel{\text{def}}{=} C$. The operational semantics of choreographies is defined as the smallest relation

$$\longrightarrow^{\mathcal{D}} \subseteq \mathcal{S} \times C \times \mathbb{R} \times \mathcal{S} \times C$$

satisfying the rules below (we omit $\mathcal{D}$ when it is not relevant):

$$\begin{array}{ll} \text{(Interact)} & (S, p \rightarrow \{p_1, \dots, p_n\} : \Sigma_{j \in J} \lambda_j : u_j; C_j) \longrightarrow_{\lambda_j} (S[u_j], C_j) \\ \text{(IfThenElseT)} & E \downarrow_S = \text{tt} \Rightarrow (S, \text{if } E@p \text{ then } C_1 \text{ else } C_2) \longrightarrow_1 (S, C_1) \\ \text{(IfThenElseF)} & E \downarrow_S = \text{ff} \Rightarrow (S, \text{if } E@p \text{ then } C_1 \text{ else } C_2) \longrightarrow_1 (S, C_2) \\ \text{(Call)} & X \stackrel{\text{def}}{=} C \in \mathcal{D} \Rightarrow (S, X) \longrightarrow_P^{\mathcal{D}} (S, C) \end{array}$$

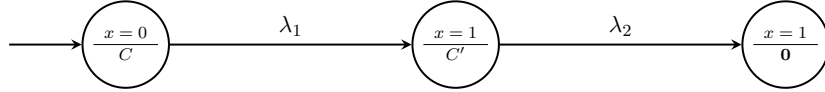
The above semantics is *fully probabilistic*: for every configuration (S, C) , the rules determine a unique probability distribution over successor configurations. In particular, there is no source of nondeterminism in the semantics. As a consequence, the transition relation $\longrightarrow$ induces a *Discrete-Time Markov Chain* (DTMC) when the labels λ_j are probabilities, or a *Continuous-Time Markov Chain* (CTMC) when they are interpreted as rates. Intuitively, the operational rules define a global (possibly infinite-state) Markov chain whose states are configurations of the form (S, C) , and whose transitions are exactly those derivable by the rules above, weighted by the corresponding probabilities or rates. This global Markov chain contains all possible choreographies of the language and all their possible states, independently of any particular initial program. Equivalently, one may consider the Markov chain induced by restricting the global chain to the set of configurations reachable from

a given pair (S, C) . Then, the behaviour of the choreography is given by the executions starting from this initial configuration, that is, by all reachable (finite or infinite) paths in the Markov chain. In the remainder, we implicitly adopt this program-indexed view.

Example 1. Consider the following choreography:

$$C = \mathbf{p} \rightarrow \{\mathbf{q}\} \lambda_1 : (x' = 1); \quad \mathbf{p} \rightarrow \{\mathbf{q}\} \lambda_2 : (x' = 1); \quad \mathbf{0}$$

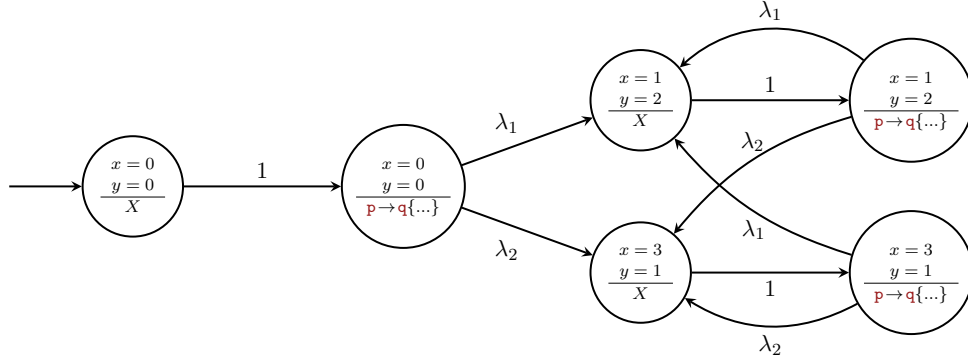
The semantics of C induces a Markov chain with states (S_0, C) , (S_1, C') , and $(S_1, \mathbf{0})$ such that $C' = \mathbf{p} \rightarrow \{\mathbf{q}\} \lambda_1 : (x' = 1); \mathbf{0}$, $S_0(x) = 0$, and $S_1(x) = 1$. The chain can be depicted as



Example 2. Consider the following choreography definition:

$$X \stackrel{\text{def}}{=} \mathbf{p} \rightarrow \{\mathbf{q}\} \begin{cases} \lambda_1 : (x' = 1) \ \& \ (y' = 2); \ X \\ \lambda_2 : (x' = 3) \ \& \ (y' = 1); \ X \end{cases}$$

Let S_0 be a state such that $S_0(x) = 0$ and $S_0(y) = 0$. Denote by S_1 and S_2 the states obtained by the two branches of the interaction, i.e., $S_1(x) = 1, S_1(y) = 2$ and $S_2(x) = 3, S_2(y) = 1$. The induced Markov chain for a recursive call X contains six states, depicted as



3.3. Other language constructs. Our language includes constructs that are not part of the formal syntax defined above. These constructs are purely syntactic sugar and can be easily encoded. Below, we discuss each of them:

- *Parametric modules.* In our implemented language, modules can be parameterised (indexed) as done in PRISM. We denote parameterised modules as $\mathbf{p}[n]$ for n ranging some finite set N . As an example, the choreography

$$p[i] \rightarrow \{q[i]\} : \lambda : U; \quad r \rightarrow \{q[i]\} : \lambda : U; \quad \mathbf{0}$$

can be easily encoded as:

$$\begin{aligned} p1 &\rightarrow \{q1\} : \lambda : U; \\ p2 &\rightarrow \{q2\} : \lambda : U; \\ p3 &\rightarrow \{q3\} : \lambda : U; \\ r &\rightarrow \{q1\} : \lambda : U; \\ r &\rightarrow \{q2\} : \lambda : U; \\ r &\rightarrow \{q3\} : \lambda : U; \quad \mathbf{0} \end{aligned}$$

Additionally, the choreography

$$p[i] \rightarrow \{q[i]\} : \lambda : U; \quad q[i+1] \rightarrow \{p[i]\} : \lambda : U; \quad \mathbf{0}$$

can be encoded in our model language as

$$\begin{aligned} p1 &\rightarrow \{q1\} : \lambda : U; \\ p2 &\rightarrow \{q2\} : \lambda : U; \\ p3 &\rightarrow \{q3\} : \lambda : U; \\ q2 &\rightarrow \{p1\} : \lambda : U; \\ q3 &\rightarrow \{p2\} : \lambda : U; \\ q1 &\rightarrow \{p3\} : \lambda : U; \quad \mathbf{0} \end{aligned}$$

Given that parameters range over a finite set, this operation is redundant.

- *The foreach construct.* The use of parametric modules can be further facilitated by introducing syntactic sugar that allows iteration over the set of indices that parameterise these modules. To this end, our language implementation includes the construct `foreach` $(k \text{ op } i) \ u@A[k]$, which can be used in front of and update u to parameterise it with respect to multiple instances of the same type of variable over different modules.

The construct enables the expression $(k \text{ op } i)$, where k binds in $u@A[k]$, to identify a set of indices which can then be used in the update $u@A[i]$. Such set is obtained from the predicate determined by the operator op with respect to the distinguished index i . Formally, $(k \text{ op } i)$ denotes the set $\{k \in I \mid op(k, i)\}$. For example, $(k \neq i)$ denotes $\{k \in I \mid k \neq i\}$, that is, all defined indices except i .

In essence, the `foreach` construct provides a concise and readable way to express operations over multiple indexed modules. It can be encoded explicitly by enumerating the modules manually, provided that the indices are known at compile time rather than determined dynamically at runtime.

- *The allsynch construct.* Our implementation supports `allsynch` $\{p_i : G_i\}_{i \in I}$, where G has the form $g \rightarrow \Sigma_{i \in I} \lambda_i : u_i$. The construct requires all modules p_i to synchronise while independently choosing a branch, unlike the interaction $p \rightarrow \{p_1, \dots, p_n\} : \Sigma_{j \in J} \lambda_j : u_j; C_j$, where the chosen branch must be taken by all modules. This provides a structured mechanism for defining interactions in which multiple modules must coordinate, while their precise behaviour may vary dynamically (with a certain probability distribution or rate). For example, consider the synchronisation between modules p and q :

$$\text{allsynch} \left\{ \begin{array}{l} p : (x = 5) \rightarrow 42 : (x' = 0) \\ p : (x = 5) \rightarrow 67 : (x' = 100) \\ p : (x = 5 \ \& \ z = 3) \rightarrow 10 : (x' = 13) \\ q : (y = 1) \rightarrow 1 : (y' = 0) \end{array} \right\}; \mathbf{0}$$

The above choreography states that (i) p and q must synchronise, and (ii) p can execute any of the three declared updates with the respective rates, provided their guards are satisfied. Such behaviour partially departs from the usual *global* description given by a choreography. In this case, there is indeed a global synchronisation between the modules, but it is not specified which command must be paired with which. In a state where $x = 5$ and $z = 3$, p can execute any of these actions. However, in a state where $x = 5$ but $z \neq 3$, only one of the first two updates can be carried out.

A key advantage of this construct is its compactness and readability in specifying this type of interactions. In the scenario above, where modules p and q participate in a

synchronised exchange, the **allsynch** syntax allows for a concise definition of conditions under which each module updates its state. This avoids the need for manually encoding synchronisation through nested conditional constructs. Despite its usefulness, the **allsynch** construct does not introduce any new expressiveness but serves as syntactic sugar for an equivalent formulation. In fact, the example above can be rewritten by nesting **if-then-else** constructs, ensuring that the synchronisation conditions are met:

```

if (x = 5 & z = 3)@p then
  if (y = 1)@q then
    p → {q} {
      42 : (x' = 0) & (y' = 0); 0
      67 : (x' = 100) & (y' = 0); 0
      10 : (x' = 13) & (y' = 0); 0
    }
  else 0
else if (x = 5 & z ≠ 3)@p then
  if (y = 1)@q then
    p → {q} {
      42 : (x' = 0) & (y' = 0); 0
      67 : (x' = 100) & (y' = 0); 0
    }
  else 0

```

The two formulations are equivalent, with the **allsynch** syntax acting as syntactic sugar that provides a structured and readable representation of synchronisation. In general, its encoding into the core choreographic language requires determining non-overlapping guards in advance. For example, from the three guards $(x = 5)$, $(x = 5)$, and $(x = 5 \ \& \ z = 3)$, it is necessary to derive the conditional guards $(x = 5 \ \& \ z = 3)$ and $(x = 5 \ \& \ z \neq 3)$. This process depends on the functions $f(\tilde{E})$ used in guards.

Note that the **allsynch** construct is more efficient (in terms of code size) than manually encoding synchronisation with nested **if-then-else** statements. As the number of synchronising modules and conditions increases (in our example, there is only a single choice for **q**), the depth and complexity of the nested conditionals grow exponentially, making the explicit formulation harder to read, write, and process. In contrast, **allsynch** provides a compact and structured way to express the same logic without the combinatorial explosion of nested conditionals.

4. THE PRISM LANGUAGE

We now give a formal definition of a fragment of the PRISM language by introducing its formal syntax and semantics.

4.1. Syntax. We reuse some of the syntactic terms used for our choreography language, including assignments and expressions. In the sequel, let a range over a (possibly infinite)

set of labels $\mathcal{L}$. We define the syntax of (a subset of) the PRISM language as follows:

(Networks)	$N, M ::=$	$\mathbf{0}$	empty network
		$ \text{ p} : \{F_i\}_i$	module
		$ M [A] M$	parallel composition
(Commands)	$F ::=$	$[\alpha] g \rightarrow \Sigma_{i \in I} \lambda_i : u_i$	$(\alpha \in \{\epsilon\} \cup \mathcal{L})$

Networks are the top syntactic category for system of modules composed together. The term $\mathbf{0}$ represent an empty network. A module is meant to represent a process running in the system and is denoted by its name and its commands, formally written as $\text{p} : \{F_i\}_i$, where p is the name and the F_i 's are commands. Networks can be composed in parallel, in a CSP style: a term like $M_1|[A]|M_2$ says that networks M_1 and M_2 can synchronise using labels in the finite set A . In this work, we omit PRISM's hiding and substitution constructs as they are irrelevant for our current choreography language. Commands in a module have the form $[\alpha]g \rightarrow \Sigma_{i \in I} \{\lambda_i : u_i\}$. The character α can either be the empty string ϵ or a label a , i.e., $\alpha \in \{\epsilon\} \cup \mathcal{L}$. If ϵ then no synchronisation is required. On the other hand, if there is label a then there will be a synchronisation with other modules that must synchronise on a . The term g is a guard on the current variable state. If both label and guard are enabled, then the command executes a branch i with probability/rate λ_i . As for choreographies, if the λ_i 's are probabilities, we must have that $0 \leq \lambda_i \leq 1$ and $\Sigma_{i \in I} \lambda_i = 1$.

4.2. Semantics. To give a probabilistic semantics to the PRISM language, we follow the approach given in the PRISM documentation [1]. Hereby, we do that by defining two relations: one with labels for networks and one on states. Our relation on networks is the smallest relation $\rightsquigarrow$ satisfying the rules given in Fig. 1. Rule (M) just exposes a command

$$\begin{array}{c}
 \frac{F \in \{F_k\}_k}{\text{p} : \{F_k\}_k \rightsquigarrow F} \text{ (M)} \quad \frac{\exists j \in \{1,2\}. M_j \rightsquigarrow [\alpha] g \rightarrow \Sigma_{i \in I} \lambda_i : u_i \quad \alpha \notin A}{M_1|[A]|M_2 \rightsquigarrow [\alpha] g \rightarrow \Sigma_{i \in I} \lambda_i : u_i} \text{ (P}_1\text{)} \\
 \\
 \frac{M_1 \rightsquigarrow [a] g \rightarrow \Sigma_{i \in I} \lambda_i : u_i \quad M_2 \rightsquigarrow [a] g' \rightarrow \Sigma_{j \in J} \lambda'_j : u'_j \quad a \in A}{M_1|[A]|M_2 \rightsquigarrow [a] g \wedge g' \rightarrow \Sigma_{i,j} \lambda_i * \lambda'_j : u_i \& u'_j} \text{ (P}_2\text{)}
 \end{array}$$

Figure 1: Semantics for PRISM networks

at network level. Rule (P₁) propagates a command through parallel composition if α is empty or if the label a is not part of the set A . When the label a is in A , we apply rule (P₂). In this case, the product of the probabilities/rates must be taken by extending the two different branches to every possible combination. This also includes the combination of the associated assignments.

Based on the relation above, given $M \rightsquigarrow [\alpha] g \rightarrow \Sigma_{i \in I} \lambda_i : u_i$ and two states S and S' , we define the function

$$\mu([\alpha] g \rightarrow \Sigma_{i \in I} \lambda_i : u_i, S, S') = \Sigma_{S[u_i]=S', i \in I} \lambda_i$$

which gives the probability/rate for the system to go from state S to state S' after executing command $[\alpha] g \rightarrow \Sigma_{i \in I} \lambda_i : u_i$, for some α . If the λ_i are probabilities, then the function must be a probability distribution. Note that $\mu(F, S, S')$ only denotes the probability/rate for the system to move from state S to state S' after executing command F . However, there

can be other commands derived from a given network M through the relation $\rightsquigarrow$ that would cause a transition from S to S' . Therefore, we define the transition relation on states $M \vdash S \rightarrow_\lambda S'$ as

$$\frac{\forall j, \alpha. M \rightsquigarrow F_j \quad S \vdash F_j}{M \vdash S \rightarrow_{\sum_j \mu(F_j, S, S')} S'} \text{ (Transition)}$$

where $S \vdash [\alpha] g \rightarrow \sum_{i \in I} \lambda_i : u_i$ is defined as $g \downarrow_S$. Note that since PRISM is declarative, a term M never changes while the state of the system evolves.

It is important to point out that, in general, the transition rule above does not give the exact probability of a transition in case of a Markov chain (DTMC), since the sum $\sum_j \mu(F_j, S, S')$ could be a value greater than 1. In order to get the right probability, the value has to be normalised for all reachable S' . In the next section, we will show that this is not an issue for networks that are obtained from our translation from choreography to PRISM.

Example 3. Consider the following network M :

$$\begin{aligned} \mathbf{p} : \{ & \quad \Box x = 0 \rightarrow 1 : (x' = 1) \\ & \quad [a] y < 1 \rightarrow 0.4 : (x' = x + 1) + 0.6 : (x' = x) \quad \} \\ \mathbf{q} : \{ & \quad \Box y = 0 \rightarrow 1 : (y' = 1) \\ & \quad [a] x < 1 \rightarrow 0.5 : (y' = y + 1) + 0.5 : (y' = y) \quad \} \end{aligned}$$

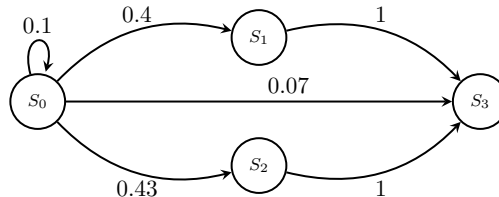
above, the two modules $\mathbf{p}$ and $\mathbf{q}$ can both do independent actions, as well as synchronising on label a . Applying the semantics, we can easily derive $M \rightsquigarrow \Box x = 0 \rightarrow 1 : (x' = 1)$, $M \rightsquigarrow \Box y = 0 \rightarrow 1 : (y' = 1)$, and $M \rightsquigarrow F$, such that

$$\begin{aligned} F = [a] x < 1 \ \& \ y < 1 \rightarrow & 0.2 : (x' = x + 1) \ \& \ (y' = y + 1) \\ & + 0.2 : (x' = x + 1) \ \& \ (y' = y) \\ & + 0.3 : (x' = x) \ \& \ (y' = y + 1) \\ & + 0.3 : (x' = x) \ \& \ (y' = y) \end{aligned}$$

Let S_0, S_1, S_2 , and S_3 be all possible reachable states such that $S_0(x) = S_0(y) = 0$, $S_1(x) = 1$, $S_1(y) = 0$, $S_2(x) = 0$, $S_2(y) = 1$, and $S_3(x) = S_3(y) = 1$. Moreover, let S_0 be the starting state. Then,

$$\begin{aligned} \mu(\Box x = 0 \rightarrow 1 : (x' = 1), S_0, S_1) &= 1 & \mu(\Box x = 0 \rightarrow 1 : (x' = 1), S_2, S_3) &= 1 \\ \mu(\Box y = 0 \rightarrow 1 : (y' = 1), S_0, S_2) &= 1 & \mu(\Box y = 0 \rightarrow 1 : (y' = 1), S_1, S_3) &= 1 \\ \mu(F, S_0, S_1) &= 0.2 & \mu(F, S_0, S_2) &= 0.3 \\ \mu(F, S_0, S_0) &= 0.3 & \mu(F, S_0, S_3) &= 0.2 \end{aligned}$$

Now, by rule (Transition) above, we have that $M \vdash S_0 \rightarrow_{1.2} S_1$, $M \vdash S_0 \rightarrow_{1.3} S_2$, $M \vdash S_0 \rightarrow_{0.2} S_3$, and $M \vdash S_0 \rightarrow_{0.3} S_0$. Clearly, both transitions should be normalised, finally yielding the following DTMC:



Example 4. The choreography presented in Example 2 can be described by the following PRISM network M (for $\lambda_i = \mu_i * \gamma_i$):

$$\begin{aligned} \mathbf{p} : \{ & [a] \ s_{\mathbf{p}} = 0 \rightarrow \mu_1 : (x' = 1) \ \& \ (s'_{\mathbf{p}} = 1) & [] \ s_{\mathbf{p}} = 1 \rightarrow 1 : (s'_{\mathbf{p}} = 0) \\ & [b] \ s_{\mathbf{p}} = 0 \rightarrow \mu_2 : (x' = 3) \ \& \ (s'_{\mathbf{p}} = 2) & [] \ s_{\mathbf{p}} = 2 \rightarrow 1 : (s'_{\mathbf{p}} = 0) \} \\ \mathbf{q} : \{ & [a] \ s_{\mathbf{q}} = 0 \rightarrow \gamma_1 : (y' = 2) \ \& \ (s'_{\mathbf{q}} = 1) & [] \ s_{\mathbf{q}} = 1 \rightarrow 1 : (s'_{\mathbf{q}} = 0) \\ & [b] \ s_{\mathbf{q}} = 0 \rightarrow \gamma_2 : (y' = 1) \ \& \ (s'_{\mathbf{q}} = 2) & [] \ s_{\mathbf{q}} = 2 \rightarrow 1 : (s'_{\mathbf{q}} = 0) \} \end{aligned}$$

The two modules $\mathbf{p}$ and $\mathbf{q}$ synchronise on the labels a and b . Applying the semantics, the global state can evolve according to F_1 or F_2 , defined as follows:

$$\begin{aligned} F_1 &= [a] \ s_{\mathbf{p}} = 0 \ \& \ s_{\mathbf{q}} = 0 \rightarrow \mu_1 * \gamma_1 : (x' = 1) \ \& \ (y' = 2) \ \& \ (s'_{\mathbf{p}} = 1) \ \& \ (s'_{\mathbf{q}} = 1) \\ F_2 &= [b] \ s_{\mathbf{p}} = 0 \ \& \ s_{\mathbf{q}} = 0 \rightarrow \mu_2 * \gamma_2 : (x' = 3) \ \& \ (y' = 1) \ \& \ (s'_{\mathbf{p}} = 2) \ \& \ (s'_{\mathbf{q}} = 2) \end{aligned}$$

5. PROJECTION

In this section, we provide a treatment of projection, which constitutes the mapping from choreographies to the PRISM language.

5.1. Mapping Choreographies to PRISM. The process of generating endpoint code from a choreography is commonly referred to as *projection*. Typically, projection is defined separately for each module appearing in the choreography program: given a module (often called a role) and a choreography, it generates the code for that particular module. In our approach, projection targets PRISM modules and is realised using the mechanisms provided by the PRISM modelling language. In particular, a choreography interaction is simulated in PRISM by using (i) labels on which the involved modules synchronise and (ii) state variables that enable the appropriate commands at the correct points in the execution.

Before formalising this idea, we introduce additional machinery that allows us to define projection. To this end, we annotate choreographies with labels. We refer to such choreographies as *annotated choreographies*. We assume two disjoint classes of labels. The first class consists of *control labels*, ranged over by a, b , which are taken from the PRISM set of labels $\mathcal{L}$. These labels will later be used as synchronisation labels in the projection to PRISM modules. The second class consists of *state labels*, ranged over by ι , drawn from some countably infinite set, subset of all possible values v in the syntactic category for expressions E . State labels represent the local control states associated with modules during the execution of the projection of a choreography.

Definition 2 (Annotated Choreography). *An annotated choreography is obtained from the choreography syntax by decorating each construct with a control label and each module occurrence with a state label:*

$$C^a ::= \mathbf{p}^{\iota} \rightarrow^a \{ \mathbf{p}_1^{\iota_1}, \dots, \mathbf{p}_n^{\iota_n} \} : \Sigma_{j \in J} \lambda_j : u_j; \ C_j^a \mid a : \text{if } E @ \mathbf{p}^{\iota} \text{ then } C_1^a \text{ else } C_2^a \mid X^{a, \sigma} \mid \mathbf{0}^{\sigma}$$

Above, σ is an injective function assigning a state label to each module. The intuition behind annotations is that they allow us to identify a particular point in the choreography, enabling the projection of all modules involved to synchronise accordingly. Fresh control labels are generated by fresh_c and fresh state labels by fresh_s . Then, we can define the function annot for annotating a choreography as follows:

Definition 3 (Annotation). *The annotation function $\text{annot}(C, \sigma) = C^a$ is defined as follows:*

$$\begin{array}{c}
\frac{a = \text{fresh}_c \quad \sigma'_i = \sigma[\mathbf{p} \mapsto \text{fresh}_s, \mathbf{p}_1 \mapsto \text{fresh}_s, \dots, \mathbf{p}_n \mapsto \text{fresh}_s] \quad \forall j \in J. \text{annot}(C_j, \sigma'_j) = C_j^a}{\text{annot}(\mathbf{p} \rightarrow \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \Sigma_{j \in J} \lambda_j : u_j; C_j, \sigma) = \mathbf{p}^{\sigma(\mathbf{p})} \rightarrow^a \{\mathbf{p}_1^{\sigma(\mathbf{p}_1)}, \dots, \mathbf{p}_n^{\sigma(\mathbf{p}_n)}\} : \Sigma_{j \in J} \lambda_j : u_j; C_j^a} \\
\\
\frac{a = \text{fresh}_c \quad \sigma_i = \sigma[\mathbf{p} \mapsto \text{fresh}_s] \quad \text{annot}(C_1, \sigma_1) = C_1^a \quad \text{annot}(C_2, \sigma_2) = C_2^a}{\text{annot}(\text{if } E @ \mathbf{p} \text{ then } C_1 \text{ else } C_2, \sigma) = \text{if } E @ \mathbf{p}^{\sigma(\mathbf{p})} \text{ then } C_1^a \text{ else } C_2^a} \\
\\
\frac{a = \text{fresh}_c}{\text{annot}(X, \sigma) = X^{a, \sigma}} \quad \frac{}{\text{annot}(\mathbf{0}, \sigma) = \mathbf{0}^\sigma}
\end{array}$$

The function above places a fresh control label a in every language construct (except from $\mathbf{0}$). Additionally, it marks each occurrence of a module $\mathbf{p}$ with the state label $\sigma(\mathbf{p})$. After a state label is used, the function updates the state assignment by replacing it with a fresh state labels in the continuations. Note that this mechanism does not make state labels globally unique, since a module may be assigned the same state label on different branches during the annotation process. For example, consider the choreography $\mathbf{p} \rightarrow \{\mathbf{q}\} : \{\lambda_1 : u_1; \mathbf{r} \rightarrow \{\mathbf{p}\} : \lambda_3 : u_3, \lambda_2 : u_2; \mathbf{r} \rightarrow \{\mathbf{p}\} : \lambda_4 : u_4\}$. After annotating the first interaction, modules $\mathbf{p}$ and $\mathbf{q}$ receive different fresh state labels in the two continuations. However, module $\mathbf{r}$ is not involved in the first interaction and therefore retains the same state label $\sigma(\mathbf{r})$ in both branches, resulting in $\mathbf{p}^{\iota_1} \rightarrow^a \{\mathbf{q}^{\iota_2}\} : \{\lambda_1 : u_1; \mathbf{r}^{\iota_3} \rightarrow^b \{\mathbf{p}^{\iota_4}\} : \lambda_3 : u_3, \lambda_2 : u_2; \mathbf{r}^{\iota_3} \rightarrow^c \{\mathbf{p}^{\iota_5}\} : \lambda_4 : u_4\}$. Each definition $X \stackrel{\text{def}}{=} C$ in $\mathcal{D}$ must be annotated as $\text{annot}(\sigma, C)$ starting from a fresh σ .

Given an annotated choreography, the partial function top returns the first state label associated with a given module in the choreography. Formally, it is defined as:

$$\begin{array}{ll}
\text{top}_{\mathbf{p}}(\mathbf{p}^{\iota} \rightarrow^a \{\mathbf{p}_1^{\iota_1}, \dots, \mathbf{p}_n^{\iota_n}\} : \Sigma_{j \in J} \lambda_j : u_j; C_j^a) = \iota & \\
\text{top}_{\mathbf{p}_i}(\mathbf{p}^{\iota} \rightarrow^a \{\mathbf{p}_1^{\iota_1}, \dots, \mathbf{p}_n^{\iota_n}\} : \Sigma_{j \in J} \lambda_j : u_j; C_j^a) = \iota_i & (1 \leq i \leq n) \\
\text{top}_{\mathbf{q}}(\mathbf{p}^{\iota} \rightarrow^a \{\mathbf{p}_1^{\iota_1}, \dots, \mathbf{p}_n^{\iota_n}\} : \Sigma_{j \in J} \lambda_j : u_j; C_j^a) = \text{top}_{\mathbf{q}}(C_1) & (\forall i, j. \text{top}_{\mathbf{q}}(C_i^a) = \text{top}_{\mathbf{q}}(C_j^a)) \\
\text{top}_{\mathbf{p}}(\text{if } E @ \mathbf{p}^{\iota} \text{ then } C_1^a \text{ else } C_2^a) = \iota & \\
\text{top}_{\mathbf{q}}(\text{if } E @ \mathbf{p}^{\iota} \text{ then } C_1^a \text{ else } C_2^a) = \text{top}_{\mathbf{q}}(C_1^a) & (\text{top}_{\mathbf{q}}(C_1^a) = \text{top}_{\mathbf{q}}(C_2^a)) \\
\text{top}_{\mathbf{p}}(X^{a, \sigma}) = \sigma(\mathbf{p}) & \text{top}_{\mathbf{p}}(\mathbf{0}^\sigma) = \sigma(\mathbf{p})
\end{array}$$

We observe that top is always defined if the annotation is obtained from annot . More precisely, given a choreography C , a module $\mathbf{p}$, and a σ , we have that there exists ι such that $\text{top}_{\mathbf{p}}(\text{annot}(C, \sigma)) = \iota$.

Given a label a and a set of indices I , the set $\{a_i | i \in I\}$ contains unique identifiers generated from a . We do similarly for state labels ι . Moreover, we extend a state with special variables $s_{\mathbf{p}}$, one for each module that we are considering in a given choreography. The intuition is to use this variable to store the state label of the choreography that its projection is simulating. This is in line with the example in Section 1, except that our formal projection uses annotation labels instead of natural numbers. We can then define the projection function. Since there are key differences between using probabilities and using rates, we proceed separately. We begin with choreographies that involve rates:

Definition 4 (Projection, CTMC). *Given an annotated choreography with rates C^a and a module $\mathbf{q}$, we define the function proj as:*

$$\text{proj}(\mathbf{q}, \mathbf{p}^\iota \rightarrow^a \{\mathbf{p}_1^{\iota_1}, \dots, \mathbf{p}_n^{\iota_n}\} : \Sigma_{j \in J} \lambda_j : u_j; C_j^a) = \boxed{\text{if } \mathbf{q} = \mathbf{p}} \\ \{[a_j] \ s_{\mathbf{q}} = \iota \rightarrow \lambda_j : s'_{\mathbf{q}} = \text{top}_{\mathbf{q}}(C_j^a) \ \& \ u_j \downarrow_{\mathbf{q}}\}_{j \in J} \cup \bigcup_{j \in J} \text{proj}(\mathbf{q}, C_j^a)$$

$$\text{proj}(\mathbf{q}, \mathbf{p}^\iota \rightarrow^a \{\mathbf{p}_1^{\iota_1}, \dots, \mathbf{p}_n^{\iota_n}\} : \Sigma_{j \in J} \lambda_j : u_j; C_j^a) = \boxed{\text{if } \mathbf{q} = \mathbf{p}_i} \\ \{[a_j] \ s_{\mathbf{q}} = \iota_i \rightarrow 1 : s'_{\mathbf{q}} = \text{top}_{\mathbf{q}}(C_j^a) \ \& \ u_j \downarrow_{\mathbf{q}}\}_{j \in J} \cup \bigcup_{j \in J} \text{proj}(\mathbf{q}, C_j^a)$$

$$\text{proj}(\mathbf{q}, \mathbf{p}^\iota \rightarrow^a \{\mathbf{p}_1^{\iota_1}, \dots, \mathbf{p}_n^{\iota_n}\} : \Sigma_{j \in J} \lambda_j : u_j; C_j^a) = \bigcup_{j \in J} \text{proj}(\mathbf{q}, C_j^a) \quad \boxed{\text{if } \mathbf{q} \notin \{\mathbf{p}, \mathbf{p}_1, \dots, \mathbf{p}_n\}}$$

$$\text{proj}(\mathbf{q}, \text{if } E @^a \mathbf{p}^\iota \text{ then } C_1^a \text{ else } C_2^a) = \boxed{\text{if } \mathbf{q} = \mathbf{p}} \\ \left\{ \begin{array}{l} \square \ s_{\mathbf{q}} = \iota \ \& \ E \rightarrow 1 : s'_{\mathbf{q}} = \text{top}_{\mathbf{q}}(C_1^a), \\ \square \ s_{\mathbf{q}} = \iota \ \& \ \text{not}(E) \rightarrow 1 : s'_{\mathbf{q}} = \text{top}_{\mathbf{q}}(C_2^a) \end{array} \right\} \cup \text{proj}(\mathbf{q}, C_1^a) \cup \text{proj}(\mathbf{q}, C_2^a)$$

$$\text{proj}(\mathbf{q}, \text{if } E @^a \mathbf{p}^\iota \text{ then } C_1^a \text{ else } C_2^a) = \text{proj}(\mathbf{q}, C_1^a) \cup \text{proj}(\mathbf{q}, C_2^a) \quad \boxed{\text{if } \mathbf{q} \neq \mathbf{p}}$$

$$\text{proj}(\mathbf{q}, \mathbf{0}^\sigma) = \emptyset \quad \text{proj}(\mathbf{q}, X^{a, \sigma}) = \{[a] \ s_{\mathbf{q}} = \sigma(\mathbf{q}) \rightarrow 1 : s'_{\mathbf{q}} = \text{top}_{\mathbf{q}}(C^a)\} \quad \boxed{X \stackrel{\text{def}}{=} C^a \in \mathcal{D}}$$

We examine the various cases in the definition above. Intuitively, the projection function makes sense only for choreographies that have been annotated using **annot**. The first three cases deal with the projection of an interaction. When projecting the first module $\mathbf{p}$, we create one command $[a_j] \ s_{\mathbf{q}} = \iota \rightarrow \lambda_j : s'_{\mathbf{q}} = \text{top}_{\mathbf{q}}(C_j^a) \ \& \ u_j \downarrow_{\mathbf{q}}$ for each branch such that

- the label a_j and its uniqueness ensure that all modules take the same branch;
- the guard $s_{\mathbf{q}} = \iota$ ensures that these commands are only executable when the system has reached this particular state, identified by the reserved variable $s_{\mathbf{q}}$;
- the rate λ_j is the rate that appears in the same branch of the choreography
- the successor state is determined by updating $s_{\mathbf{q}}$ to $\text{top}_{\mathbf{q}}(C_j^a)$, depending on which branch j was selected. The function **annot** ensures that every step in all branches is assigned to a different label, thereby also discarding all branches that are not selected;
- the projected update $u_j \downarrow_{\mathbf{q}}$ acts as a filter on the list of updates in u_j , ensuring that only those variables local to $\mathbf{q}$ are updated. We do not define $u_j \downarrow_{\mathbf{q}}$ explicitly, but we assume that it adheres to the allocation of variables fixed prior to projection. Since PRISM would reject an input in which multiple modules update the same variables, our projection is undefined in such cases.

The second case defines the projection of an interaction for one of the modules $\{\mathbf{p}_1, \dots, \mathbf{p}_n\}$. Similarly to the previous case, we define a command for each branch of the interaction. However, the rate of each command is set to 1, ensuring that each branch synchronises with rate $\lambda_j \cdot 1$ (see rule (P_2) in Figure 1). The third case concerns the projection of a module that does not belong to the set $\{\mathbf{p}, \mathbf{p}_1, \dots, \mathbf{p}_n\}$. In this case, the projection

collects the commands generated from all C_j^a . Note that **annot** assigns the same state label: this is not a problem, since any synchronisation is performed on different control labels. The if-then-else construct focuses on the module **p**, where the guard E is evaluated. Being this a local operation, no synchronisation labels are required. For recursive calls, we generate a command that redirects the execution to the projection of the definitions $\mathcal{D}$, which are projected separately.

Example 5. *In order to illustrate how our projection works, consider the following example, in which we apply the projection to the choreography X from Example 2. This yields PRISM modules that are slightly different from those presented in Example 4, but which implement the same behaviour. In Example 2, we defined a recursive choreography in which module **p** interacts with module **q** through two branches. Its annotated form can be written as:*

$$X \stackrel{\text{def}}{=} \mathbf{p}^{\iota_1} \rightarrow^a \{ \mathbf{q}^{\iota_2} \} \left\{ \begin{array}{l} \lambda_1 : (x' = 1) \& (y' = 2); \quad X^{b, \sigma_1} \\ \lambda_2 : (x' = 3) \& (y' = 1); \quad X^{c, \sigma_2} \end{array} \right.$$

where $\sigma_1 = \sigma[\mathbf{p} \mapsto \text{fresh}_{\mathbf{c}}, \mathbf{q} \mapsto \text{fresh}_{\mathbf{c}}]$ for some σ such that $\sigma(\mathbf{p}) = \iota_1$ and $\sigma(\mathbf{q}) = \iota_2$. Our projection, as defined in Definition 4, when applied to a given choreography $X^{d, \sigma'}$ (which essentially corresponds to a recursive call), generates the following modules:

$$\begin{aligned} \mathbf{p} : & \left\{ \begin{array}{l} [d] \ s_{\mathbf{p}} = \sigma'(\mathbf{p}) \rightarrow 1 : (s'_{\mathbf{p}} = \iota_1), \\ [a_1] \ s_{\mathbf{p}} = \iota_1 \rightarrow \lambda_1 : (x' = 1) \& (s'_{\mathbf{p}} = \sigma_1(\mathbf{p})), \\ [a_2] \ s_{\mathbf{p}} = \iota_1 \rightarrow \lambda_2 : (x' = 3) \& (s'_{\mathbf{p}} = \sigma_2(\mathbf{p})), \end{array} \right. & \left\{ \begin{array}{l} [b] \ s_{\mathbf{p}} = \sigma_1(\mathbf{p}) \rightarrow 1 : (s'_{\mathbf{p}} = \iota_1), \\ [c] \ s_{\mathbf{p}} = \sigma_2(\mathbf{p}) \rightarrow 1 : (s'_{\mathbf{p}} = \iota_1) \end{array} \right\} \\ \mathbf{q} : & \left\{ \begin{array}{l} [d] \ s_{\mathbf{q}} = \sigma'(\mathbf{q}) \rightarrow 1 : (s'_{\mathbf{q}} = \iota_2), \\ [a_1] \ s_{\mathbf{q}} = \iota_2 \rightarrow 1 : (y' = 2) \& (s'_{\mathbf{q}} = \sigma_1(\mathbf{q})), \\ [a_2] \ s_{\mathbf{q}} = \iota_2 \rightarrow 1 : (y' = 1) \& (s'_{\mathbf{q}} = \sigma_2(\mathbf{q})), \end{array} \right. & \left\{ \begin{array}{l} [b] \ s_{\mathbf{q}} = \sigma_1(\mathbf{q}) \rightarrow 1 : (s'_{\mathbf{q}} = \iota_2), \\ [c] \ s_{\mathbf{q}} = \sigma_2(\mathbf{q}) \rightarrow 1 : (s'_{\mathbf{q}} = \iota_2) \end{array} \right\} \end{aligned} \quad (5.1)$$

The core idea is that we use reserved variables $s_{\mathbf{p}}$ and $s_{\mathbf{q}}$ to track the state of each module, in order to simulate the behaviour of the choreography. The various steps of the projection can then be summarised as follows:

- (1) We first project $X^{d, \sigma'}$. This generates the commands $[d] \ s_{\mathbf{p}} = \sigma'(\mathbf{p}) \rightarrow 1 : (s'_{\mathbf{p}} = \iota_1)$ and $[d] \ s_{\mathbf{q}} = \sigma'(\mathbf{q}) \rightarrow 1 : (s'_{\mathbf{q}} = \iota_2)$ for **p** and **q**, respectively. Since $X^{d, \sigma'}$ is the starting choreography, we assume that all PRISM modules initially start in a state where the reserved variables satisfy $s_{\mathbf{p}} = \sigma'(\mathbf{p})$ and $s_{\mathbf{q}} = \sigma'(\mathbf{q})$. The intention is that the commands of the two modules synchronise on d and then activate the first action in the body of X .
- (2) Separately, we must project the bodies of all definitions in $\mathcal{D}$, only one in this case. The first step of the projection ensures that this interaction is enabled only when the variables $s_{\mathbf{p}}$ and $s_{\mathbf{q}}$ are equal to the label ι_1 and ι_2 respectively. The operation has two possible outcomes: either a synchronisation on label a_1 with rate λ_1 , or a synchronisation on label a_2 with rate λ_2 . In the first case, both $s_{\mathbf{p}}$ and $s_{\mathbf{q}}$ are set to $\sigma_1(\mathbf{p})$ and $\sigma_1(\mathbf{q})$, respectively. In the second case, $s_{\mathbf{p}}$ and $s_{\mathbf{q}}$ are respectively set to $\sigma_2(\mathbf{p})$ and $\sigma_2(\mathbf{q})$.

The projection from Definition 4 works only with rates. In the case of probabilities, we must ensure that the probabilities of a branching sum to 1, which is not guaranteed by Definition 4. This is because we cannot enforce both **p** and $\{\mathbf{p}_1, \dots, \mathbf{p}_n\}$ to take the same branch according to the probability distribution given by the λ_i 's. To address this issue, we introduce the following definition instead:

Definition 5 (Projection, DTMC). *Given an annotated choreography with probabilities C^a and a module $\mathbf{q}$, we define proj as:*

$$\begin{aligned} \text{proj}(\mathbf{q}, \mathbf{p}^\iota \rightarrow^a \{\mathbf{p}_1^{\iota'_1}, \dots, \mathbf{p}_n^{\iota'_n}\} : \Sigma_{j \in J} \lambda_j : u_j; C_j^a) = & \boxed{\text{if } \mathbf{q} = \mathbf{p}} \\ \left\{ \begin{array}{l} \square \ s_{\mathbf{q}} = \iota_1 \rightarrow \Sigma_{j \in J} \lambda_j : s'_{\mathbf{q}} = \iota_2 \\ [a_j] \ s_{\mathbf{q}} = \iota_2 \rightarrow \lambda_j : s'_{\mathbf{q}} = \text{top}_{\mathbf{q}}(C_j^a) \ \& \ u_j \downarrow_{\mathbf{q}} \end{array} \right\}_{j \in J} \cup \bigcup_{j \in J} \text{proj}(\mathbf{q}, C_j^a) \end{aligned}$$

The other cases of the definition are equivalent to those in Definition 4.

The main intuition behind the above definition is that module $\mathbf{p}$ takes a (probabilistic) internal decision on the j^{th} branch and then synchronises on label a_j with $\{\mathbf{p}_1, \dots, \mathbf{p}_n\}$.

5.2. Correctness. Our projection guarantees a correspondence between the semantics of a choreography and that of its projection. Below, we assume to work with annotated choreographies. However, to make notation lighter, we do not use annotation when not necessary. In order to state our main theorem, we rely on the notions of *head modules* and *strongly connected* choreographies. The former identifies the modules involved in the next action of a choreography:

Definition 6 (Head Modules). *The function hMods is defined as follows:*

$$\begin{aligned} \text{hMods}(\mathbf{p} \rightarrow \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \Sigma_{j \in J} \lambda_j : u_j; C_j) &= \{\mathbf{p}, \mathbf{p}_1, \dots, \mathbf{p}_n\} \\ \text{hMods}(\text{if } E @ \mathbf{p} \text{ then } C_1 \text{ else } C_2) &= \{\mathbf{p}\} \\ \text{hMods}(X) &= \text{hMods}(C) \quad (\text{if } X \stackrel{\text{def}}{=} C \in \mathcal{D}) \\ \text{hMods}(\mathbf{0}) &= \emptyset \end{aligned}$$

Then, the property of strongly connected is defined below.

Definition 7 (Strongly Connected Choreography). *A choreography C is strongly connected, written $\text{sConn}(C)$, if it satisfies the following conditions:*

$$\begin{aligned} \frac{\forall j \in J. \text{sConn}(C_j) \wedge \text{hMods}(C_j) \neq \emptyset \Rightarrow \text{hMods}(C_j) \cap \{\mathbf{p}, \mathbf{p}_1, \dots, \mathbf{p}_n\} \neq \emptyset}{\text{sConn}(\mathbf{p} \rightarrow \{\mathbf{p}_1, \dots, \mathbf{p}_n\} : \Sigma_{j \in J} \lambda_j : u_j; C_j)} \quad \overline{\text{sConn}(\mathbf{0})} \\ \frac{\forall j \in \{1, 2\}. \text{sConn}(C_j) \wedge \text{hMods}(C_j) \neq \emptyset \Rightarrow \mathbf{p} \in \text{hMods}(C_j)}{\text{sConn}(\text{if } E @ \mathbf{p} \text{ then } C_1 \text{ else } C_2)} \quad \frac{\text{sConn}(C) \quad X \stackrel{\text{def}}{=} C \in \mathcal{D}}{\text{sConn}(X)} \end{aligned}$$

The notion of connectedness is quite well-known in the literature. Since our framework is based on synchronous communication, we follow the same approach as that of Carbone et al. [7]. The basic idea is that each interaction shares at least one module with the subsequent choreography. In particular, in every branch of a probabilistic choice or an if-then-else involving modules $\mathbf{p}_1, \dots, \mathbf{p}_n$, the first action (if any) of every other module $\mathbf{q}$ must be an interaction with one of $\mathbf{p}_1, \dots, \mathbf{p}_n$, possibly after unfolding recursive calls. Connectedness guarantees a form of causality between interactions: subsequent interactions between modules always depend on previous ones. As an example of strongly connected choreography, the choreography X , obtained from the definition

$$X \stackrel{\text{def}}{=} \mathbf{p} \rightarrow \{\mathbf{q}\} : \left(\begin{array}{l} \lambda_1 : u_1; \ \mathbf{q} \rightarrow \{\mathbf{r}\} : \lambda'_1 : u'_1; \ X \\ \lambda_2 : u_2; \ X \end{array} \right)$$

is strongly connected while $\mathbf{p} \rightarrow \{\mathbf{q}\} : (\lambda_1 : u_1; \mathbf{r}_1 \rightarrow \{\mathbf{r}_2\} : \lambda'_1 : u'_1; \mathbf{0})$ is not. Note, it is possible to transform choreographies that are not strongly connected into strongly connected ones, by adding extra interactions.

We are now ready to state our main theorem, which relates a (projectable, strongly connected) choreography with its PRISM projection. In the sequel, S_+ denotes the extension of a global state S with the additional reserved variables $s_{\mathbf{q}}$ (one for each module $\mathbf{q}$), introduced by the projection. The projection uses alphabetised parallel composition $\parallel$, where modules synchronise only on shared labels.

Theorem 1 (Projection). *Let C be a choreography such that $\text{sConn}(C)$, let $C^a = \text{annot}(C, \sigma)$ for some fresh σ , and let S_+ be an extension of a global state S such that for all $\mathbf{q} \in C$, we have that $S_+(s_{\mathbf{q}}) = \text{top}_{\mathbf{q}}(C^a)$. Then,*

$$(S, C^a) \longrightarrow_{\lambda} (S', C_1^a) \text{ if and only if } \parallel_{\mathbf{q} \in C} \text{proj}(\mathbf{q}, C^a) \vdash S_+ \longrightarrow_{\lambda} S'_+,$$

such that:

- for all $\mathbf{q} \in C^a$, we have that $\text{proj}(\mathbf{q}, C_1^a) \subseteq \text{proj}(\mathbf{q}, C^a)$
- $S' = S'_+ \setminus \{s_{\mathbf{q}}\}_{\mathbf{q} \in C^a}$
- for all $\mathbf{q} \in C^a$, we have that $S'_+(s_{\mathbf{q}}) = \text{top}_{\mathbf{q}}(C_1^a)$

Proof. The proof proceeds by induction on C . We remove annotations when not relevant, in order to make the presentation clearer.

- $C^a = \mathbf{p}^{\iota} \rightarrow^a \{\mathbf{p}_1^{\iota_1}, \dots, \mathbf{p}_n^{\iota_n}\} : \sum_{j \in J} \lambda_j : u_j; C_j^a$. By the only applicable rule (Interact), we have

$$(S, \mathbf{p}^{\iota} \rightarrow^a \{\mathbf{p}_1^{\iota_1}, \dots, \mathbf{p}_n^{\iota_n}\} : \sum_{j \in J} \lambda_j : u_j; C_j^a) \longrightarrow_{\lambda_k} (S[u_k], C_k).$$

By definition of projection, we obtain the following PRISM commands. Module $\mathbf{p}$ is projected as:

$$\{[a_j] \ s_{\mathbf{q}} = \iota \rightarrow \lambda_j : s'_{\mathbf{q}} = \text{top}_{\mathbf{q}}(C_j^a) \ \& \ u_j \downarrow_{\mathbf{q}}\}_{j \in J} \cup \bigcup_{j \in J} \text{proj}(\mathbf{q}, C_j^a)$$

Modules $\mathbf{p}_i$ are projected as:

$$\{[a_j] \ s_{\mathbf{q}} = \iota_i \rightarrow 1 : s'_{\mathbf{q}} = \text{top}_{\mathbf{q}}(C_j^a) \ \& \ u_j \downarrow_{\mathbf{q}}\}_{j \in J} \cup \bigcup_{j \in J} \text{proj}(\mathbf{q}, C_j^a)$$

Any other role $\mathbf{q}$ is projected as:

$$\bigcup_{j \in J} \text{proj}(\mathbf{q}, C_j^a)$$

We must show two properties: first, that the projection above can make the same transition; and second, that if the projection makes a transition, it necessarily corresponds to the transition of the choreography above. Observe that, since our annotated choreography is obtained from `annot`, the state of the generated CTMC is uniquely identified by the labels $\iota, \iota_1, \dots, \iota_n$ that can only synchronise on one of the labels a_j . Any other command among these modules must be enabled by a different state label.

Therefore, the only-if direction of the theorem follows directly by applying rule (Interact), whose premise is satisfied by deriving, via rule (P₂), the command:

$$\begin{aligned} [a_k] \ (s_{\mathbf{p}} = \iota) \wedge (s_{\mathbf{p}_1} = \iota_1) \wedge \dots \wedge (s_{\mathbf{p}_n} = \iota_n) \rightarrow \\ (\lambda_k \cdot 1^n) : \ (u_k \downarrow_{\mathbf{p}}) \ \& \ (u_k \downarrow_{\mathbf{p}_1}) \ \& \ \dots \ \& \ (u_k \downarrow_{\mathbf{p}_n}) \\ \ \& \ (s'_{\mathbf{p}} = \text{top}_{\mathbf{p}}(C_k^a)) \ \& \ (s'_{\mathbf{p}_1} = \text{top}_{\mathbf{p}_1}(C_k^a)) \ \& \ \dots \ \& \ (s'_{\mathbf{p}_n} = \text{top}_{\mathbf{p}_n}(C_k^a)) \end{aligned}$$

For the converse direction, we must show that the above is the only enabled reduction in the PRISM semantics. As for modules $\mathbf{p}, \mathbf{p}_1, \dots, \mathbf{p}_n$, since the state S_+ yields $\iota, \iota_1, \dots, \iota_n$, no other reduction is possible. Other modules, projected as $\bigcup_{j \in J} \text{proj}(\mathbf{q}, C_j^a)$, are locally enabled by state S_+ to perform their next action. If, by contradiction, we can apply rule (Transition), derived by an application of rule (P₁) on some label b (which must be different from a), on a different set of modules $\{\mathbf{r}_k\}$, then we could show that there is no causal dependency between modules $\mathbf{p}, \mathbf{p}_1, \dots, \mathbf{p}_n$ and modules $\{\mathbf{r}_k\}$, violating indeed the fact that the choreography is strongly connected. This reasoning is standard for projection theorems like the one we are proving [7].

- $C^a = \text{if } E @^a \mathbf{p} \text{ then } C_1^a \text{ else } C_2^a$. In this case, the projection of $\mathbf{p}$ is

$$\left\{ \begin{array}{l} \parallel s_{\mathbf{q}} = \iota \ \& \ E \rightarrow 1 : s'_{\mathbf{q}} = \text{top}_{\mathbf{q}}(C_1^a), \\ \parallel s_{\mathbf{q}} = \iota \ \& \ \text{not}(E) \rightarrow 1 : s'_{\mathbf{q}} = \text{top}_{\mathbf{q}}(C_2^a) \end{array} \right\} \cup \text{proj}(\mathbf{q}, C_1^a) \cup \text{proj}(\mathbf{q}, C_2^a)$$

while, for all other modules, we have $\text{proj}(\mathbf{q}, C_1^a) \cup \text{proj}(\mathbf{q}, C_2^a)$. In this case, module $\mathbf{p}$ is enabled by the uniqueness of ι , while all other modules are not simply because we assume that the choreography is strongly connected. Hence, any other synchronisation or if-then-else statement is blocked, as it must involve $\mathbf{p}$.

- $C^a = X^{a,\sigma}$. By definition, assuming $X \stackrel{\text{def}}{=} C_X \in \mathcal{D}$, it must be the case that, for all $\mathbf{q} \in C$:

$$\text{proj}(\mathbf{q}, X^{a,\sigma}) = \{[a] \ s_{\mathbf{q}} = \sigma(\mathbf{q}) \rightarrow 1 : s'_{\mathbf{q}} = \text{top}_{\mathbf{q}}(C_X^a)\} \cup \bigcup_{(Y \stackrel{\text{def}}{=} C_Y) \in \mathcal{D}} (\text{proj}(\mathbf{q}, C_Y))$$

In the only-if direction, we know that $(S, X^{a,\sigma}) \rightarrow_1 (S, C_X^a)$. From the uniqueness of a , we can derive $\parallel_{\mathbf{q} \in C} \text{proj}(\mathbf{q}, C) \vdash S_+ \rightarrow_\lambda S'_+$ by applying rule (M) on each module, then rule (P₂), and finally rule (Transition). Moreover, we observe that the command $s'_{\mathbf{q}} = \text{top}_{\mathbf{q}}(C_X^a)$ has no effect on the state except modifying the extended variables with the top annotation in C_X^a which, by assumption again, it is unique. Additionally, we observe that for all $\mathbf{q} \in C^a$, we have that $\text{proj}(\mathbf{q}, C_1^a) \subseteq \text{proj}(\mathbf{q}, C^a)$. Finally, for all $\mathbf{q} \in C$, we have that $S'_+(s_{\mathbf{q}}) = \text{top}_{\mathbf{q}}(C_1^a)$.

In the other direction, we just observe that, for all $\mathbf{q}$, the extended state variables $s_{\mathbf{q}}$ are all set to $\sigma(\mathbf{q})$. Therefore, given uniqueness of annotations, the projected modules can only simulate the procedure call.

- $C = \mathbf{0}^\sigma$. Immediate.

The case for DTMC is also similar. □

We observe that the theorem departs from standard projection results for choreography languages. In most settings, both choreographies and endpoints are expressed in a process algebra in which control flow is encoded syntactically, and correctness is shown by relating reductions of programs. PRISM, instead, is declarative: a module consists of a fixed set of commands, and execution is driven solely by state changes through the enabling of commands. Our choreography language combines both aspects. Although it maintains an explicit runtime state for variables, its syntax determines the next action to be executed. The projection reconciles this mismatch by encoding control-flow information into the state: the variables $s_{\mathbf{q}}$ act as a distributed program counter, yielding the extended state S_+ . Consequently, choreography reduction corresponds in the projection to transitions on S_+ rather than to syntactic changes of PRISM modules.

Remark 1. *Our theorem induces an asymmetry: projecting after reducing a choreography may result in fewer commands than projecting first and then reducing. This is because PRISM execution never alters the set of commands. Let us focus on an example where we compare the semantics of the choreography $X^{d,\sigma'}$, obtained from the recursive definition in Example 2, with that of its projection from Example 5. Starting from a state S_0 in which $x = 0$ and $y = 0$, and from its extension S_0^+ , where additionally $s_{\mathbf{p}} = \sigma'(\mathbf{p})$ and $s_{\mathbf{q}} = \sigma'(\mathbf{q})$, both the choreography and its projection from Equation 5.1 have a single possible transition:*

$$(S_0, X^{d,\sigma'}) \longrightarrow_1 \left(S_0, \mathbf{p}^{\iota_1} \rightarrow^a \{\mathbf{q}^{\iota_2}\} \left\{ \begin{array}{l} \lambda_1 : (x' = 1) \ \& \ (y' = 2); \ X^{b,\sigma_1} \\ \lambda_2 : (x' = 3) \ \& \ (y' = 1); \ X^{c,\sigma_2} \end{array} \right. \right)$$

$$\|_{\mathbf{r} \in X^d} \text{proj}(\mathbf{r}, X^{d,\sigma'}) \vdash S_+ \longrightarrow_\lambda S_+[s'_{\mathbf{p}} = \iota_1][s'_{\mathbf{q}} = \iota_2]$$

Clearly, the projection of the unfolding is strictly contained in the projection of $X^{d,\sigma'}$. This is because the commands $[d] \ s_{\mathbf{p}} = \sigma'(\mathbf{p}) \rightarrow 1 : (s'_{\mathbf{p}} = \iota_1)$ and $[d] \ s_{\mathbf{q}} = \sigma'(\mathbf{q}) \rightarrow 1 : (s'_{\mathbf{q}} = \iota_2)$ are no longer generated. However, the PRISM model obtained by projecting the reductum exhibits the same behaviour as the original one. Relating these two states would require a suitable notion of equivalence induced by the semantics of PRISM, which we leave as future work. We conjecture that, under an appropriate equivalence notion, such as probabilistic bisimulation, our theorem could be extended to capture this additional feature.

6. BENCHMARKING

To evaluate our approach, we implemented the projection from choreographies to PRISM in Java, using ANTLR to parse our language and automatically generate the corresponding PRISM modules. This implementation realises the projection formally defined in Section 5 and ensures that each benchmark presented below is automatically produced from a single global choreography specification. The tool thus serves as an executable validation of our formal framework: it guarantees that the semantics of the generated PRISM models coincide with those of the original choreographies and provides the basis for the empirical analysis reported in this section.

In the following, we present an experimental evaluation of our language. The examples highlight two main points: (i) the choreographic representation is significantly more concise than the corresponding PRISM code, and (ii) PRISM exhibits equivalent behaviour on both the projected and original models. In this context, "equivalent behaviour" refers to agreement on the quantitative properties that the original case studies were designed to analyse, such as reachability and time-bounded probabilities or expected values. Our projection targets a structured fragment of PRISM and is not intended to reproduce the exact syntactic form or the full range of nondeterminism present in some hand-written models. As a consequence, we do not aim for bisimilarity or semantic equivalence at the model level. Instead, we verify that the projected and reference models lead to the same results for the relevant benchmark properties, which is the criterion of equivalence we adopt in this evaluation.

We focus on six benchmarks: a modified version of the example reported in Section 2, a simple peer-to-peer protocol, the Bitcoin Proof-of-Work protocol [5], the Hybrid Casper protocol [13], a synchronous leader-election protocol [17], and a modelisation of the dining cryptographers [11]. The generated PRISM files are available in our online repository [2].

A Modified thinkteam Protocol. In this modified version of the thinkteam protocol introduced in the earlier sections, we extend the protocol to involve generalised interactions with possible many receivers. Specifically, the `CheckOut` process now communicates with two users simultaneously, `User1` and `User2` each tasked with performing distinct actions upon access to the file.

In the first branch, `User1` increments the variable `x` by 1, while `User2` decrements the variable `y` by 1. Conversely, in the second branch, the roles are reversed, with `User1` decrementing `x` and `User2` incrementing `y`.

```

C0 := CheckOut → User1, User2 : (+["1*lambda"] " " "(x=x+1)" "(y=y-1)"; C1
                                +["1*lambda"] " " "(x=x-1)" "(y=y+1)"; C2)
C1 := CheckOut → User1, User2 : (+["1*theta"] ; C0)
C2 := CheckOut → User1, User2 : (+["1*mu"] ; C1 +["1*mu"] ; C2)

```

Listing 1: Choreography for the Modified thinkteam Protocol

Part of the generated PRISM model is reported in Listing 2. It presents the behaviour in a decentralised form: interactions are encoded through synchronising commands distributed across the relevant modules. As a result, understanding the overall flow requires considering how these module-local transitions align, rather than reading a single global specification as in the choreographic representation.

```

1  module CheckOut
2    CheckOut.STATE : [0..2] init 0;
3    [MMHOL] (CheckOut.STATE=0) -> 1 : (CheckOut.STATE'=1);
4    [FFSFW] (CheckOut.STATE=0) -> 1 : (CheckOut.STATE'=2);
5    [ULCFN] (CheckOut.STATE=1) -> 1 : (CheckOut.STATE'=0);
6    [YHHWG] (CheckOut.STATE=2) -> 1 : (CheckOut.STATE'=1);
7    [XWSAO] (CheckOut.STATE=2) -> 1 : (CheckOut.STATE'=2);
8  endmodule
9  ...
10 module User2
11   User2.STATE : [0..2] init 0;
12   [MMHOL] (User2.STATE=0) -> lambda : (y'=y-1)&(User2.STATE'=1);
13   [FFSFW] (User2.STATE=0) -> lambda : (y'=y+1)&(User2.STATE'=2);
14   [ULCFN] (User2.STATE=1) -> mu : (User2.STATE'=0);
15   [YHHWG] (User2.STATE=2) -> theta : (User2.STATE'=1);
16   [XWSAO] (User2.STATE=2) -> theta : (User2.STATE'=2);
17 endmodule

```

Listing 2: Part of the generated PRISM model for the Modified thinkteam Protocol

Simple Peer-To-Peer Protocol. This case study describes a simple peer-to-peer protocol based on BitTorrent². The model comprises a set of clients trying to download a file that has been partitioned into K blocks. Initially, there is one client that has already obtained all of the blocks and N additional clients with no blocks. Each client can download a block from any of the others but they can only attempt four concurrent downloads for each block. The code we analyze with $K = 5$ and $N = 4$ is reported in Listing 3.

```

PeerToPeer := Client[i] → Client[i]: (+["rate1*1"] "(b[i]1'=1)"&&" " . PeerToPeer
                                +["rate2*1"] "(b[i]2'=1)"&&" " . PeerToPeer

```

²<https://www.prismmodelchecker.org/casestudies/peer2peer.php>

```

+["rate3*1"] "(b[i]3'=1)"&&" " . PeerToPeer
+["rate4*1"] "(b[i]4'=1)"&&" " . PeerToPeer
+["rate5*1"] "(b[i]5'=1)"&&" " . PeerToPeer

```

Listing 3: Choreography for the Peer-To-Peer Protocol

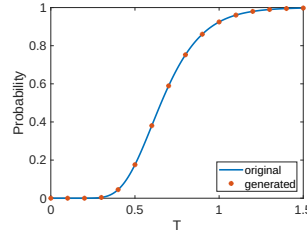
```

1  module Client1
2      Client1 : [0..1] init 0;
3      b11 : [0..1];
4      b12 : [0..1];
5      b13 : [0..1];
6      b14 : [0..1];
7      b15 : [0..1];
8
9      [] (Client1=0) → rate1 : (b11'=1)&(Client1'=0);
10     [] (Client1=0) → rate2 : (b12'=1)&(Client1'=0);
11     [] (Client1=0) → rate3 : (b13'=1)&(Client1'=0);
12     [] (Client1=0) → rate4 : (b14'=1)&(Client1'=0);
13     [] (Client1=0) → rate5 : (b15'=1)&(Client1'=0);
14 endmodule

```

Listing 4: Part of the generated PRISM program for the Peer-To-Peer Protocol

Part of the generated PRISM code is shown in Listing 4 and it is faithful with what is reported in the PRISM documentation. In Figure 2, we compare the probabilities that all clients (in a model with 4 clients: **Client1**, ..., **Client4**) have received all blocks within the time interval $0 \leq T \leq 1.5$, as obtained from both our generated model and the model reported in the documentation. This property serves as a benchmark to evaluate whether the generated model preserves the expected behaviour of the original specification. In this case, there are no differences in the results or the time required to compute the property.

Figure 2: Probability that clients received all the blocks before time T , with $0 \leq T \leq 1.5$

Proof of Work Bitcoin Protocol. In [5], the authors extended the PRISM model checker syntax to incorporate dynamic data types, enhancing its capabilities to model the Proof of Work protocol used in the Bitcoin blockchain [24].

```

PoW := Hasher[i] → Miner[i] :
(+["mR*hr[i]"] " " "(b[i]'=createB(b[i],B[i],c[i]))&(c[i]'=c[i]+1)" ;
  Miner[i] → Network : ("rB*1" "(B[i]'=addBlock(B[i],b[i]))"
    foreach(k!=i) "(set[k]'=addBlockSet(set[k],b[i]))"@Network;PoW)
+["lR*hr[i]"] ;
  if "!isEmpty(set[i])"@Miner[i] then {
    ["r"] "(b[i]'=extractBlock(set[i]))"@Miner[i] ;

```

```

    Miner[i] → Network : ([ "1*1" ] "(setMiner[i]'=addBlockSet(setMiner[i],b[i]))"
                                "(set[i]' = removeBlock(set[i],b[i]))";PoW)
  }
  else{
    if "canBeInserted(B[i],b[i])"@Miner[i] then {
      [ "1" ] "(B[i]'=addBlock(B[i],b[i]))&(setMiner[i]'=removeBlock(setMiner[i],b[i]))"@Miner[i];
      PoW
    }
    else{PoW}
  }
}

```

Listing 5: Choreography for the Proof of Work Bitcoin Protocol

In summary, the code depicts miners engaging in solving PoW, updating their ledgers, and communicating with the network. The indices i represent the module renaming feature of the choreographic language. Thus, each interaction will be repeated for each miner and hasher of the protocol that we are considering. The protocol works as follows. When synchronising with the hasher, a miner tries to solve a cryptographic puzzle. Successful attempts add a new block to its ledger and update other miners' block sets. Unsuccessful attempts involve extracting a block, updating its ledger and block sets, and continuing the PoW process.

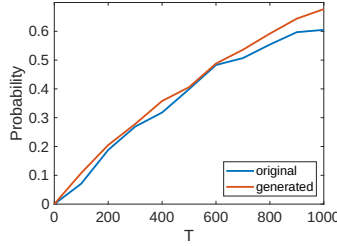


Figure 3: Probability that a block is created within T time units, $0 \leq T \leq 1000$

The PRISM model we created is more verbose than the one in [5], mainly because we consistently generate the else branch for if-then-else expressions, resulting in a higher number of instructions. Despite this, the experimental results for block creation probability within a bound time T (Figure 3) remain unaffected. Any discrepancies between the original and generated models are due to inherent variations in the simulation-based calculation of probability.

Hybrid Casper Protocol. We now present the Hybrid Casper Protocol [13]. The Hybrid Casper protocol represents a hybrid consensus protocol for blockchains, merging features from both Proof of Work and Proof of Stake protocols.

```

PoS := Hasher[i] → Validator[i] :
  (+["mR*1"] "(b[i]'=createB(b[i],L[i],c[i]))&(c[i]'=c[i]+1)";
    if "!(mod(getHeight(b[i]),EpochSize)=0)"@Validator[i] then{...}
    else{
      Validator[i] → Vote_Manager : ([ "1*1" ] "(Votes'=addVote(Votes,b[i],stake[i]))"; PoS)}
  +["hR*1"] ; if "isEmpty(set[i])"@Validator[i] then { ... }
    else{ PoS }
  +["rC*1"] "(lastCheck[i]'=extractCheckpoint(listCheckpoints[i],lastCheck[i]))"...

```

Listing 6: Excerpt of choreography for the Hybrid Casper Protocol

The modelling approach is very similar to the one used for the Proof of Work Bitcoin protocol. Specifically, the Hybrid Casper protocol is represented in PRISM as the parallel composition of n `Validator` modules, along with the modules `Vote_Manager` and `Network`. Each `Validator` module closely resembles the `Miner` module from the previous protocol. The module `Vote_Manager` is responsible for storing maps containing votes for each block and computing associated rewards/penalties.

The choreographic model for this example is reported in Listing 6. The code resembles that of the Proof of Work protocol, but each validator can either create a new block, receive blocks from the network module, or determine if it is eligible to vote for specific blocks. For lack of space, we detailed only part of the code, the complete model can be found in [2].

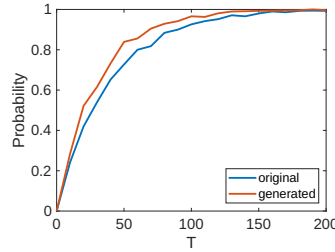


Figure 4: Probability that a block is created within T time units, $0 \leq T \leq 200$

The generated code is very similar the one outlined in [13], with the main distinction being the greater number of lines in our generated model. This difference is due to the fact that certain commands could be combined, but our generation lacks the automatic capability to perform this check. While the results obtained for the probability of creating a block within the time T reported in Figure 4 exhibit similarity, running simulations for the generated model takes PRISM 39.016 seconds, compared to the 22.051 seconds required for the original model.

Synchronous Leader Election. This case study examines the synchronous leader election protocol proposed by Itai & Rodeh [17], designed to elect a leader in a ring of N processors by exchanging messages. The protocol operates in rounds, where each processor selects a random ID from $\{1, \dots, K\}$, circulates it around the ring, and determines if a unique maximum ID exists. If so, the processor with this ID becomes the leader; otherwise, the process repeats in the next round.

For illustration, we considered the case where $N = 4$ and $K = 8$, following the PRISM model³. We modeled this example in our choreographic language, as shown in Listing 7, capturing the protocol's behaviour and dynamics.

```

Election := allSynch{ j in [1..4]
    Process[j] : (true -> "1/K" : "(p[i] != 0) & (v[i] != 0) & (u[i] == true)" +
    ... + "1/K" : "(p[i] == 0) & (v[i] == 7) & (u[i] == true)" ) }.
allSynch{
    Counter : ("(c < N-1)" -> "1" : "(c' = c+1)")

```

³https://www.prismmodelchecker.org/casestudies/synchronous_leader.php

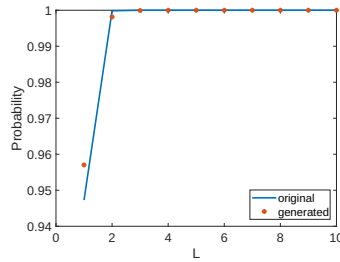
```

Counter : (c=N-1) -> "1" : "(c'=c)"
Process1 : ("u1&!(p1=2)&(c<N-1)" -> "1" : "(u1'=true)&(v1'=v2)")
Process1 : ("u1&(p1=2)&(c<N-1)" -> "1" : "(u1'=false)&(v1'=v2)&(p1'=0)")
Process1 : ("!u1&(c<N-1)" -> "1" : "(u1'=false)&(v1'=v2)")
Process1 : ("u1&!(p1=v2)&(c=N-1)" -> "1" : "(u1'=true)&(v1'=0)&(p1'=0)")
Process1 : ("u1&(p1=v2)&(c=N-1)" -> "1" : "(u1'=false)&(v1'=0)&(p1'=0)")
Process1 : ("!u1&(c=N-1)" -> "1" : "(u1'=false)&(v1'=0)")
...
}.
if "u1 | u2 | u3 | u4"@Counter then {
  Counter -> Process[i] : ([ "1*1" ] "(c'=c)" "(u[i]'=false)&(v[i]'=0)&(p[i]'=0)".
  allSynch {
    Counter : (true -> "1" : "(c'=c)")
    Process1 : (true -> "1" : " ")
    ...
  } . END)
}
else{ Counter -> Process[i] : ([ "1*1" ] "(c'=1)" "(u[i]'=false)&(v[i]'=0)&(p[i]'=0)" . Election)}

```

Listing 7: Choreography for the Synchronous Leader Election Protocol

While the generated model (Listing 8) successfully replicates the functionality of the PRISM repository model, a key difference lies in the modular structure of the two representations. Specifically, the generated model adopts a simplified modular design by grouping transitions more compactly in certain modules, such as the **Counter** module. This simplification reduces redundancy and may improve readability without altering the correctness or outcomes of the protocol. Importantly, this structural refinement does not impact the behaviour of the system, as the generated model remains functionally equivalent to the original PRISM repository model, as displayed in Figure 5.

Figure 5: The probability of electing a leader within L rounds, with $1 \leq L \leq 10$

```

1  module Counter
2    Counter : [0..4] init 0;
3    c : [0..N-1] init 0;
4    [YQBDX] (Counter = 0)&(c<N-1) -> 1 : (c'=c+1)&(Counter'=1);
5    [YQBDX] (Counter = 0)&(c=N-1) -> 1 : (c'=c)&(Counter'=1);
6    [ELTMI] (Counter=1)&(u1 | u2 | u3 | u4) -> 1 : (c'=c)&(Counter'=2);
7    [LJTIP] (Counter=1)&!(u1 | u2 | u3 | u4) -> 1 : (c'=1)&(Counter'=0);
8    [AWUQP] (Counter = 2)-> 1 : (c'=c)&(Counter'=2);
9  endmodule
10 module Process1
11 Process1 : [0..4] init 0;
12 p1 : [0..K-1] init 0;
13 v1 : [0..K-1] init 0;

```

```

14      u1 : bool;
15      [BKKXT] (Process1 = 0) -> 1/K: (p1'=0) & (v1'=0) & (u1'=true) & (Process1'=1)
16          + 1/K: (p1'=1) & (v1'=1) & (u1'=true) & (Process1'=1)
17          + 1/K: (p1'=2) & (v1'=2) & (u1'=true) & (Process1'=1)
18          + 1/K: (p1'=2) & (v1'=3) & (u1'=true) & (Process1'=1)
19          + 1/K: (p1'=2) & (v1'=4) & (u1'=true) & (Process1'=1)
20          + 1/K: (p1'=2) & (v1'=5) & (u1'=true) & (Process1'=1)
21          + 1/K: (p1'=2) & (v1'=6) & (u1'=true) & (Process1'=1)
22          + 1/K: (p1'=2) & (v1'=7) & (u1'=true) & (Process1'=1);
23      [YQBDX] (Process1 = 1) & u1 & (p1=2) & (c < N-1) -> 1 : (u1'=true) & (v1'=v2) & (Process1'=2);
24      [YQBDX] (Process1 = 1) & u1 & (p1=2) & (c < N-1) -> 1 : (u1'=false) & (v1'=v2) & (p1'=0) & (Process1'=2);
25      [YQBDX] (Process1 = 1) & !u1 & (c < N-1) -> 1 : (u1'=false) & (v1'=v2) & (Process1'=2);
26      [YQBDX] (Process1 = 1) & u1 & (p1=v2) & (c=N-1) -> 1 : (u1'=true) & (v1'=0) & (p1'=0) & (Process1'=2);
27      [YQBDX] (Process1 = 1) & u1 & (p1=v2) & (c=N-1) -> 1 : (u1'=false) & (v1'=0) & (p1'=0) & (Process1'=2);
28      [YQBDX] (Process1 = 1) & !u1 & (c=N-1) -> 1 : (u1'=false) & (v1'=0) & (p1'=0) & (Process1'=2);
29      [ELTMI] (Process1=2) -> 1 : (u1'=false) & (v1'=0) & (p1'=0) & (Process1'=3);
30      [LJTIP] (Process1=2) -> 1 : (u1'=false) & (v1'=0) & (p1'=0) & (Process1'=0);
31      [AWUQP] (Process1 = 3) -> 1 : (Process1'=4);
32  endmodule
33  ...

```

Listing 8: Part of the generated PRISM model for the Synchronous Leader Election Protocol

Dining Cryptographers. The generated model for this example does not faithfully model the original one. We chose to include it in the paper to demonstrate the limitations of our approach, specifically in cases where the abstraction may not fully capture the behaviour of the original protocol. This allows us to analyze and understand where discrepancies may arise, highlighting areas for improvement in the model generation process. This case study explores the dining cryptographers protocol introduced by Chaum [11], which allows a group of N cryptographers to determine whether their master has anonymously paid for dinner without revealing the identity of the payer. The protocol functions by having each cryptographer flip a fair coin and share the outcome with their right-hand neighbour. Each cryptographer then publicly declares whether the two coins they observe—one they flipped and one received from the left—match or differ. If a cryptographer is the payer, they deliberately alter their response. The final count of “agree” statements follows a predictable pattern: for an odd number of cryptographers, an odd count indicates that one of them paid, while an even count means the master paid. This pattern reverses for an even number of participants.

To illustrate the protocol, we examined the case where $N = 3$, using the PRISM model⁴ reported in the official repository. We expressed this scenario in our choreographic language, as shown in Listing 9.

```

Crypto := if "(coin[i]=0)"@crypt[i] then {
    crypt[i] -> crypt[i] : (+["0.5*1"] "(coin[i]'=1)" . Crypto2
                        +["0.5*1"] "(coin[i]'=2)" . Crypto2)
}
else{ Crypto }

Crypto2 := if "((coin[i]>0)&(coin[i+1]>0))"@crypt[i] then{
    if "(coin[i]=coin[i+1])"@crypt[i] then {
        if "(pay=p[i])"@crypt[i] then { Crypto3 }
    }
}

```

⁴https://www.prismmodelchecker.org/casestudies/dining_cryptographers.php

```

        else{ ["1"] "(agree[i] '=1)"@crypt[i]. Crypto3 }
      }
    else{
      if "(pay=p[i])"@crypt[i] then { ["1"] "(agree[i] '=1)"@crypt[i]. Crypto3 }
      else{ Crypto3 }
    }
  }
else { Crypto2 }

Crypto3 := allSynch{ j in [1...3]
  crypt[j] : (true -> "1" : "true;" )
}.END

```

Listing 9: Choreography for the Dining Cryptographers Protocol

This code defines a choreography model using three distinct choreographies (`Crypto`, `Crypto2`, and `Crypto3`) to avoid redundancy and optimise the code structure. By using separate choreographies, we can reuse common logic without rewriting the entire code. The initial probabilistic branching demonstrates the use of recursion, where both branches of the probabilistic transition ultimately lead to the same state.

In fact, in the generated PRISM model in Listing 10, we can observe that from the initial state `crypt1 = 0`, we transition to two possible branches where `crypt1 = 1` regardless of whether `coin1` takes the value 1 or 2. This is a result of the recursion in the choreography, where both branches follow the same recursive path that leads to `crypt1 = 1`. The transitions in the model are shown in the following PRISM code, where both transitions have a 50% probability of either setting `coin1 = 1` or `coin1 = 2`, and both lead to the same updated state `crypt1 = 1`. Note that the code provided here only shows a part of the generated PRISM model. The code for `crypt2` and `crypt3` follows the same structure.

```

1  ...
2  module crypt1
3  crypt1 : [0..2] init 0;
4  coin1 : [0..2] init 0;
5  s1 : [0..1] init 0;
6  agree1 : [0..1] init 0;
7  [] (crypt1=0)&((coin1=0)) -> 0.5 : (coin1'=1)&(crypt1'=1)+0.5 : (coin1'=2)&(crypt1'=1);
8  [] (crypt1=0)&!((coin1=0)) -> 1:(crypt1'=0);
9  [] (crypt1=1)&(coin1>0)&(coin2>0)&((coin1=coin2))&!((pay=p1)) -> 1:(agree1'=1)&(crypt1'=2);
10 [] (crypt1=1)&(coin1>0)&(coin2>0)&((coin1=coin2))&((pay=p1)) -> 1:(crypt1'=2);
11 [] (crypt1=1)&(coin1>0)&(coin2>0)&!((coin1=coin2))&((pay=p1)) -> 1:(agree1'=1)&(crypt1'=2);
12 [] (crypt1=1)&(coin1>0)&(coin2>0)&!((coin1=coin2))&!((pay=p1))-> 1:(crypt1'=2);
13 [LERZX] (crypt1 = 2) -> 1:(crypt1'=2);
14 endmodule
15 ...

```

Listing 10: Part of the generated PRISM model for the Dining Cryptographers Protocol

In this case, however, the generated model does not exhibit the same behaviour as the original one in the PRISM repository. When analysing the probability of anonymity, the result obtained by the original model, as reported on the website, is 0.25, whereas in our generated model the probability is 0. This discrepancy arises from a structural difference in how transitions are enabled in the two models. In the original PRISM model, the initial coin toss of cryptographer 1 is specified as

$\square \text{ (coin1} = 0) \rightarrow 0.5 : (\text{coin1}' = 1) \ \& \ (\text{crypt1}' = 1) + 0.5 : (\text{coin1}' = 2) \ \& \ (\text{crypt1}' = 1);$

and is enabled solely on the condition `coin1 = 0`, independently of the current value of `crypt1`. In contrast, in our generated model the corresponding transition is guarded by both `coin1 = 0` and `crypt1 = 0` (see line 7 of Listing 10). This additional guard is not incidental: it reflects a fundamental property of our projection, namely that every interaction in a choreography is associated with a unique control state of each role. The projection introduces control variables to ensure that, for each module, only the commands corresponding to the current control state are enabled. As a result, our translated models enforce a single enabled action per module per state, whereas the original PRISM model permits the initial coin toss to be enabled in several control contexts.

This mismatch in the enabling conditions reduces the reachable state space of the generated model and leads to different quantitative results (95 states and 194 transitions in our model versus 380 states and 776 transitions in the original one). More generally, this illustrates a limitation of our current projection approach: PRISM models in which certain probabilistic transitions are enabled independently of the logical control flow of a process – potentially coexisting with other enabled commands – cannot be reproduced exactly via projection from a choreography, because the projection enforces a control discipline absent in such models.

Code Size Comparison. To complement the qualitative discussion of the benchmarks, we provide a quantitative comparison of the size of each choreography and the corresponding generated PRISM models. Table 1 summarises the number of lines of code (LoC) taken from the listings shown in this article. The measurements reflect only the fragments printed in the paper and exclude all variable declarations in both the choreography and the corresponding PRISM models. Moreover, complete PRISM models are typically larger, as each interaction from the global choreography expands into multiple module-specific commands.

Benchmark	Choreography LoC	PRISM LoC
Modified thinkteam protocol	4	~15
Peer-to-Peer protocol	6	~20
Bitcoin Proof-of-Work	~45	~100
Hybrid Casper	~70	>220
Leader Election	~20	~60
Dining Cryptographers	~17	>40

Table 1: Size comparison between choreographies and generated PRISM models.

The table highlights a structural difference between the two formalisms: choreographies describe global behaviour in a single specification, whereas PRISM distributes behaviour across several modules using synchronising commands. As a result, choreography descriptions tend to be more compact, while the generated PRISM models are necessarily more verbose in order to express the equivalent endpoint behaviour. This comparison motivates the use of choreographies as a higher-level modelling notation, while still enabling automated generation of analysable PRISM code.

7. RELATED WORK AND DISCUSSION

Related Work. Choreographic programming [22] is a language paradigm for specifying the expected interactions (communications) of a distributed system from a global viewpoint, from which decentralised implementations can be generated via projection. The notion of choreography has been substantially explored in the last decade, both from a theoretical perspective, e.g., [7, 8], to full integration into fully-fledged programming languages, such as WS-CDL [16] and Choral [14]. Nevertheless, there is a scarcity of research on probabilistic aspects of choreographic programming. To the best of our knowledge, Aman and Ciobanu [3, 4] are the only ones who studied the concept of choreography and probabilities. Their work augments multiparty session types (type abstractions for communicating systems that use the concept of choreography) with a probabilistic internal choice similar to the one used by our choreographic branching. However, they do not provide any semantics with state in terms of Markov chains, and, most importantly, they do not project into a probabilistic declarative language model such as PRISM. Carbone et al. [6] define a logic for expressing properties of a session-typed choreography language. However, the logic is undecidable and has no model-checking algorithm. As far as our knowledge extends, there is currently no work that generates probabilistic models from choreographic languages that can be then model-checked.

A complementary line of research concerns the implementability and realisability of global specifications, where the central question is whether a given global transition system can be decomposed into synchronously communicating local components whose composition preserves the intended behaviour [10, 23, 26, 27]. In those settings, the global description is typically unconstrained and substantial effort goes into characterising which behaviours admit a distributed realisation. By contrast, choreographies impose a structured form of global specification in which all interactions are explicitly declared. This discipline ensures that every choreography is directly realisable via projection, but it also restricts the kinds of distributed behaviours that can be expressed, placing our approach within a well-behaved subspace of the broader realisability landscape.

Discussion and Future Work. The ultimate goal of the proposed framework is to use the concept of choreographic programming to improve several aspects, including usability, correctness, and efficiency in modelling and analysing systems. In this paper, we address usability and efficiency of modelling systems by proposing a probabilistic choreography language. Our language improves the intuitive modelling of concurrent probabilistic systems. Traditional modelling languages often lack the expressive clarity needed to effectively capture the intricacies of such systems. By designing a language specific for choreographing system behaviours, we provide an intuitive means of specifying system dynamics. This approach enables a more natural and straightforward modelling process, essential for accurately representing real-world systems and ensuring the efficacy of subsequent analysis. Although choreographies and the projection function aim to abstract away low-level details, providing instead a higher-level representation of system behaviours, the choreographic approach naturally imposes some restrictions on expressivity. A choreography prescribes a single global interaction structure that determines, for each role, a unique enabled action at any control state. This discipline is crucial for correctness-by-construction but also limits the class of systems that can be expressed: models that rely on multiple independently enabled transitions, nondeterministic interleavings, or asynchronous local evolution cannot, in general, be captured by our current framework. The systems supported by our approach are therefore

those whose behaviour can be organised around explicitly coordinated interactions with a globally determined branching structure. Some of the case studies presented in the PRISM documentation [1] cannot be modeled by using our current language. Specifically, there are two main cases where our approach encounters limitations: *(i)* in the asynchronous leader election case study, our language prohibits the use of an “if-then” statement without an accompanying “else” to prevent deadlocked states; *(ii)* in probabilistic broadcast protocols or cyclic server polling system models, the system requires probabilistic branching to synchronise different modules based on the selected branch. These issues could be fixed by extending our choreographic language further and are therefore left as future work.

Additionally to these extensions, we conjecture that our semantics for choreographies may be used for improving performance by directly generating a CTMC or a DTMC from a choreography, bypassing the projection into the PRISM language. In fact, the Markov chain construction from a choreography seems to be faster than the construction from a corresponding projection in the PRISM language, since it is not necessary to take into account all the possible synchronisations in the rules from Fig. 1. A formal complexity analysis, an implementation, and performance benchmarking are left as future work.

In conclusion, this paper has introduced a framework for addressing the challenges of modelling and analysing concurrent probabilistic systems. The development of a choreographic language with tailored syntax and semantics offers an intuitive modelling approach. We have established the correctness of a projection function that translates choreographic models to PRISM-compatible formats. Additionally, our compiler enables seamless translation of choreographic models to PRISM, facilitating powerful analysis while maintaining expressive clarity. These contributions bridge the gap between high-level modelling and robust analysis in probabilistic systems, paving the way for advancements in the field.

REFERENCES

- [1] Prism documentation. <https://www.prismmodelchecker.org/>. Accessed: 2023-09-05.
- [2] Repository. <https://github.com/adeleveschetti/ChoreoPRISM>. Accessed: 2024-01-31.
- [3] Bogdan Aman and Gabriel Ciobanu. Probabilities in session types. In Mircea Marin and Adrian Craciun, editors, *Proceedings Third Symposium on Working Formal Methods, FROM 2019, Timișoara, Romania, 3-5 September 2019*, volume 303 of *EPTCS*, pages 92–106, 2019.
- [4] Bogdan Aman and Gabriel Ciobanu. Interval probability for sessions types. In Agata Ciabatonni, Elaine Pimentel, and Ruy J. G. B. de Queiroz, editors, *Logic, Language, Information, and Computation - 28th International Workshop, WoLLIC 2022, Iași, Romania, September 20-23, 2022, Proceedings*, volume 13468 of *Lecture Notes in Computer Science*, pages 123–140. Springer, 2022.
- [5] Stefano Bistarelli, Rocco De Nicola, Letterio Galletta, Cosimo Laneve, Ivan Mercanti, and Adele Veschetti. Stochastic modeling and analysis of the bitcoin protocol in the presence of block communication delays. *Concurr. Comput. Pract. Exp.*, 35(16), 2023.
- [6] Marco Carbone, Davide Grohmann, Thomas T. Hildebrandt, and Hugo A. López. A logic for choreographies. In Kohei Honda and Alan Mycroft, editors, *Proceedings Third Workshop on Programming Language Approaches to Concurrency and communication-cEntric Software, PLACES 2010, Paphos, Cyprus, 21st March 2010*, volume 69 of *EPTCS*, pages 29–43, 2010.
- [7] Marco Carbone, Kohei Honda, and Nobuko Yoshida. Structured communication-centered programming for web services. *ACM Trans. Program. Lang. Syst.*, 34(2):8:1–8:78, 2012.
- [8] Marco Carbone and Fabrizio Montesi. Deadlock-freedom-by-design: multiparty asynchronous global programming. In Roberto Giacobazzi and Radhia Cousot, editors, *The 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL ’13, Rome, Italy - January 23 - 25, 2013*, pages 263–274. ACM, 2013.
- [9] Marco Carbone and Adele Veschetti. A probabilistic choreography language for PRISM. In Ilaria Castellani and Francesco Tiezzi, editors, *Coordination Models and Languages - 26th IFIP WG 6.1*

- International Conference, COORDINATION 2024, Held as Part of the 19th International Federated Conference on Distributed Computing Techniques, DisCoTec 2024, Groningen, The Netherlands, June 17-21, 2024, Proceedings*, volume 14676 of *Lecture Notes in Computer Science*, pages 20–37. Springer, 2024.
- [10] Ilaria Castellani, Madhavan Mukund, and P. S. Thiagarajan. Synthesizing distributed transition systems from global specifications. In C. Pandu Rangan, V. Raman, and R. Ramanujam, editors, *Foundations of Software Technology and Theoretical Computer Science*, pages 219–231, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.
 - [11] David Chaum. The dining cryptographers problem: Unconditional sender and recipient untraceability. *Journal of Cryptology*, 1:65–75, 1988.
 - [12] Frits Dannenberg, Marta Kwiatkowska, Chris Thachuk, and Andrew Turberfield. DNA walker circuits: computational potential, design, and verification. In D. Soloveichik and B. Yurke, editors, *Proc. 19th International Conference on DNA Computing and Molecular Programming (DNA 19)*, volume 8141 of *LNCS*, pages 31–45. Springer, 2013.
 - [13] Letterio Galletta, Cosimo Laneve, Ivan Mercanti, and Adele Veschetti. Resilience of hybrid casper under varying values of parameters. *Distributed Ledger Technol. Res. Pract.*, 2(1):5:1–5:25, 2023.
 - [14] Saverio Giallorenzo, Fabrizio Montesi, and Marco Peressotti. Choral: Object-oriented choreographic programming. *ACM Trans. Program. Lang. Syst.*, 46(1):1:1–1:59, 2024.
 - [15] John Heath, Marta Kwiatkowska, Gethin Norman, David Parker, and Oksana Tymchyshyn. Probabilistic model checking of complex biological pathways. In C. Priami, editor, *Proc. Computational Methods in Systems Biology (CMSB’06)*, volume 4210 of *Lecture Notes in Bioinformatics*, pages 32–47. Springer Verlag, 2006.
 - [16] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Web services, mobile processes and types. *Bull. EATCS*, 91:160–185, 2007.
 - [17] Alon Itai and Michael Rodeh. Symmetry breaking in distributed networks. *Information and Computation*, 88(1), 1990.
 - [18] Marta Kwiatkowska, Gethin Norman, and David Parker. Probabilistic verification of herman’s self-stabilisation algorithm. *Formal Aspects of Computing*, 24(4):661–670, 2012.
 - [19] Marta Kwiatkowska, Gethin Norman, David Parker, and Maria Grazia Vigliotti. Probabilistic mobile ambients. *Theoretical Computer Science*, 410(12–13):1272–1303, 2009.
 - [20] Marta Kwiatkowska, Gethin Norman, and Roberto Segala. Automated verification of a randomized distributed consensus protocol using Cadence SMV and PRISM. In G. Berry, H. Comon, and A. Finkel, editors, *Proc. 13th International Conference on Computer Aided Verification (CAV’01)*, volume 2102 of *LNCS*, pages 194–206. Springer, 2001.
 - [21] Marta Kwiatkowska, Gethin Norman, and Jeremy Sproston. Probabilistic model checking of deadline properties in the IEEE 1394 FireWire root contention protocol. *Formal Aspects of Computing*, 14(3):295–318, 2003.
 - [22] Fabrizio Montesi. *Introduction to Choreographies*. Cambridge University Press, 2023.
 - [23] Madhavan Mukund. *From Global Specifications to Distributed Implementations*, pages 19–35. Springer US, Boston, MA, 2002.
 - [24] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. <https://bitcoin.org/bitcoin.pdf>, 2008.
 - [25] Gethin Norman and Vithaly Shmatikov. Analysis of probabilistic contract signing. *Journal of Computer Security*, 14(6):561–589, 2006.
 - [26] Maurice H. ter Beek, Rolf Hennicker, and José Proença. Realisability of global models of interaction. In Erika Ábrahám, Clemens Dubslaff, and Silvia Lizeth Tapia Tarifa, editors, *Theoretical Aspects of Computing – ICTAC 2023*, pages 236–255, Cham, 2023. Springer Nature Switzerland.
 - [27] Maurice H. ter Beek, Rolf Hennicker, and José Proença. Team automata: Overview and roadmap. In Ilaria Castellani and Francesco Tiezzi, editors, *Coordination Models and Languages*, pages 161–198, Cham, 2024. Springer Nature Switzerland.
 - [28] Maurice H. ter Beek, Mieke Massink, Diego Latella, Stefania Gnesi, Alessandro Forghieri, and Maurizio Sebastianis. Model checking publish/subscribe notification for thinkteam[®]. In *FMICS*, volume 133 of *Electronic Notes in Theoretical Computer Science*, pages 275–294. Elsevier, 2004.